

AI-Driven Intrusion Detection Systems (IDS) for
Enhanced Network Security: Improving Performance
and Evaluation Metrics of Existing AI-Driven IDS,
Explainability of Existing AI-Driven IDS, and Enabling
Universal Implementation of Newer AI-Driven IDS
based on Improving Performance and Evaluation
Metrics of Existing AI-Driven IDS
CMP7200: Individual Master's Project Dissertation

Student ID: 24175318

2025

Abstract

This Project Dissertation showcases an in-depth AI- Driven IDS framework that brings together Deep Learning (CNN, LSTM Models), Reinforcement Learning (DQN), and Hybrid DL-RL Models with the aim of improving detection accuracy, latency resilience, and generalisability across heterogeneous network datasets (most notably NSL-KDD, CIC-IDS2018, CIC-DDoS2019); this Project Dissertation detailing a rigorous and robust CRISP-DM/SDLC-based methodology (data harmonisation, SMOTEENN, Correlation Filtering, RFECV, PCA, Isolation Forest), implementing model optimisation and practical Edge-Aware design choices, and integrating xAI techniques (SHAP, LIME), also Vibe Coding LLM assisted for faster prototyping while emphasising ethical, fairness and regulatory considerations (GDPR, EU AI Act) with evaluation that uses stratified K-fold benchmarking (Accuracy, Precision, Recall, F1, AUC, Latency, Explainability Metrics) and demonstrates that Hybrid DL-RL Models incorporated with Explainable AI (xAI) that offers promising progress within this field, but still faces hurdles in computational cost, interpretability, and Edge Deployability-topics emphasised on for future improvement and model compression/federated strategies to enable practical deployment. **Keywords: AI-driven IDS; Deep Learning; Reinforcement Learning; Hybrid DL-RL; Explainable AI (xAI); Vibe Coding; Edge Computing; CRISP-DM; NSL-KDD; CIC-IDS2018; CIC-DDoS2019.**

Contents

1	Introduction	1
1.1	Background Context	1
1.2	Problem Statement and Research Questions	1
1.3	Aim and Objectives	2
1.4	Scope and Considerations	3
1.5	Structure of this Project Dissertation	3
2	Systematic Literature Review	4
2.1	Introduction of the Systematic Literature Review	4
2.2	DL for IDS	4
2.3	RL for IDS	4
2.4	Hybrid Models for IDS	5
2.5	xAI, Bias, and Transparency	5
2.6	Vibe Coding for IDS	6
2.7	Edge Computing and Resource Limitations	6
2.8	Methodological Frameworks for IDS	7
2.9	Transparency, Fairness, and Accountability for AI within IDS	7
2.10	Summary of the Literature Review	8
3	Methodology Used	8
3.1	Datasets Used	8
3.1.1	Justification of multiple Datasets Used	10
3.1.2	Advantages of Multiple Datasets Used	11
3.1.3	Disadvantages of Multiple Datasets Used	11
3.1.4	Opposing Academic Perspectives	11
3.2	Data Pre-processing Steps Used	11
3.3	Models Used and their Limitations Explained In More Detail	14
3.4	CRISP-DM Flowchart, CRISP-DM, and SDLC Methodological Frameworks and Methodology Explained In More Detail	18
3.4.1	Business Understanding	19
3.4.2	Data Understanding	20
3.4.3	Data Preparation	20
3.4.4	Modelling	20
3.4.5	Evaluation	20
3.4.6	Deployment Potential	21
4	Implementation and Evaluation of Results	22
4.1	Implementation	22
4.1.1	Importing the required Python Libraries	22
4.1.2	Loading the Acquired Datasets	23
4.1.3	Data Pre-processing: Aligning Columns, Mapping Labels, Dataset Ver- tical Aggregation, Removing Redundant Columns, Preservation of Timestamps, One-Hot Encoding of each Dataset Categorical Columns, One-Hot En- coding all of the Labels, Sorting the Datasets, Saving the final combined Dataset	24
4.1.4	Data Pre-processing: EDA	25
4.1.5	Data Pre-processing: Imputation, Feature Scaling, and Log Transform- ation on Scaled Features	26

4.1.6	Data Pre-processing: SMOTEENN	27
4.1.7	Data Pre-processing: Pearson and Spearman Correlation Filtering . . .	27
4.1.8	Data Pre-processing: RFECV	28
4.1.9	Data Pre-processing: PCA	29
4.1.10	Data Pre-processing: Isolation Forest	29
4.1.11	CNN Model	30
4.1.12	LSTM Model	31
4.1.13	DQN Model	31
4.1.14	CNN-DQN Model	32
4.1.15	LSTM-DQN Model	33
4.1.16	xAI and LLM Integration	34
4.2	Evaluation of the Results	35
5	Discussion	36
6	Future Work to Incorporate Suggestions % Conclusions	36
6.1	Future Work to Incorporate Suggestions	36
6.2	Conclusions	37

1 Introduction

1.1 Background Context

As the world is becoming digitised, Cyberattacks are increasingly becoming more dynamic, evolving into persistent, complex threats that traditional IDS struggle to detect. IDS systems with rigorous rules typically fail against zero-day exploits, adaptable malicious software (malware), and adversarial attacks. To combat this, researchers are using AI, especially DL, RL, and hybrid DL-RL models for constructing scalable and adaptable IDS that easily recognises patterns that are not recognisable as highlighted by (Doost et al. 2025; Ren et al. 2023).

Strickland, Zakar et al. (2024) note that hybrid DL-RL models further enhance and optimise this capability by amalgamating pattern recognition with making decisions. In addition xAI methods and tools such as SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) boosts interpretability and transparency, enabling humans to interpret convoluted model results(Ribeiro et al. 2016b). The recent emergence of Vibe Coding Technology as termed by Karpathy (2024) for LLMs has created an opportunity to test IDS optimally and efficiently.

In addition, recent advances in adversarial ML further complicate the Cybersecurity situation, highlighting critical limitations in signature-based and even early anomaly-based detection systems. Gen. AI is primarily used by perpetrators to conceal malicious payloads and systematically bypass traditional IDS. As noted by Zhang et al. (2023), static rule-based IDS are typically often incapable of generalising from sparse or novel attack data. In contrast, DL models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) demonstrate better performance in detecting complex spatial patterns in high-dimensional network traffic. RL agents further enhance this by dynamically adapting to changing attack surfaces via continuous human-in-the-loop feedback (Chen et al. 2023). As a result, AI- Driven IDS can now respond autonomously, making them significantly more resilient to advanced persistent threats (APTs) and zero-day exploits(Eke et al. 2023).

Furthermore, the combination of xAI with IDS is an emerging focal research area, especially in areas focused on safety such as healthcare and industrial controlled systems. As these AI systems advance, trust becomes essential-xAI, xAI techniques such as SHAP and LIME have been extended to network security, ensuring auditability, compliance, and human-in-the-loop verification (Charmet et al. 2022; Houssel et al. 2024). The hybrid DL-RL-xAI architecture not only provides detection rates with high accuracy but also intuitive, real-time threat interpretability. The incorporation of Vibe Coding as coined by Karpathy (2024), provides a new lens for developing transformer-based IDS agents that align with the real-time reasoning capabilities of large language models (LLMs). This shift in thinking is supported by Edge Computing and access to the masses with or without prior knowledge Song and Ma (2024), which sets the standard for decentralised, adaptive, and secure IDS architectures fit for deployment across dynamic enterprise and IoT environments.

1.2 Problem Statement and Research Questions

Even though there has been huge progress in AI for IDS, existing IDS infrastructures still experience limitations in detection accuracy, adaptability to new and emerging threats, and deployability across varied environments. DL models, although powerful, often have many hidden multi-layers that are complex, making transparency difficult, while RL approaches have issues of training stability and reward design. Hybrid models combining DL and RL show promise, but introduce high computational complexity and reduced interpretability.

Furthermore, the prominence of using AI-Driven IDS has been beset by a lack of standardised performance metrics, limited trust due to explainability gaps, and insufficient consideration of ethical and legal compliance, especially in real-time or edge-computing environments. The incorporation of Vibe Coding, xAI methods such as SHAP and LIME, and Edge Computing are still not explored enough in unified, scalable IDS solutions. This project dissertation is set out to address these gaps through a robust, explainable, and universally adaptable AI-driven IDS framework.

Since there are these problems, the following research questions naturally arise and need to be solved in order to enhance this topic, and to potentially solve it, as follows:

1. How exactly can performance (accuracy, latency, generalisation) be enhanced of existing AI-driven IDS models using hybrid DL-RL architecture?
2. Exactly how can xAI techniques such as SHAP and LIME be optimally incorporated to increase transparency and trust in AI-driven IDS?
3. How effective can Vibe Coding be in accelerating development and improving accessibility for non-experts in IDS construction and testing?
4. Exactly what are the practical challenges and solutions for deploying AI-driven IDS across cloud-edge infrastructures, particularly in constrained IoT environments?
5. How can standardised evaluation metrics and CRISP-DM (Cross-Industry Standard Process for Data Mining)/SDLC (Software Development Lifecycle) methodological frameworks be used to ensure reproducibility, fairness, and regulatory compliance in AI-IDS development?

1.3 Aim and Objectives

This project proposal aims to create and evaluate an AI-driven IDS framework that uses DL, RL, and hybrid architectures to optimise detection performance, while simultaneously embedding explainability and ethical safeguards to ensure trustworthiness and regulatory compliance.

The Objectives are Aims explained in greater detail as follows:

- Examine recent DL, RL and hybrid DL-RL techniques and models for IDS.
- Evaluation of xAI methods, notably LIME and SHAP for interpretability of models.
- Experimentation of Vibe Coding via the use of LLMs for IDS developmental acceleration.
- Use techniques and certain structures of Python Programming language to enable Edge Computing to prototype and test IDS.
- To obtain accuracy, latency, and explainability in Datasets, most notably NSL-KDD, CIC-IDS2018, and CIC-DDoS2019.
- Applying CRISP-DM and SDLC methodological frameworks throughout the whole developmental lifecycle.

1.4 Scope and Considerations

Scope:

- **Technical Focus:** This project emphasises on the development and evaluation of hybrid DL-RL architectures for IDS, enhanced with xAI components (SHAP, LIME), Vibe Coding techniques, and Edge Computing deployment potential by using certain Python Programming Language structures and techniques (explained fully later in section 4 of this Project Dissertation).
- **Datasets Used:** Evaluation of the NSL-KDD, CIC-IDS2018, and CIC-DDoS2019 Datasets—each chosen for their unique threat characteristics, relevance, and diversity. **Implementation Platform:** All model development, evaluation, and experimentation will occur within Google Colab using Python-based frameworks (TensorFlow, Scikit-learn, etc.).
- **Model Evaluation Metrics:** Accuracy, recall, precision, F1 score, AUC (Area Under the Curve)-ROC (Receiver Operating Characteristic curve), latency, and explainability indicators will be benchmarked using Stratified K-fold cross-validation.

Considerations:

- **Ethical and Legal:** Strict adherence to GDPR and EU AI Act regulations will be guaranteed, with xAI tools included to support transparency and human-in-the-loop oversight.
- **Bias and Fairness:** Techniques such as SMOTE and fairness audits such as True Positive Rate imbalance will be implemented to assess and reduce model bias.
- **Limitations:** Edge deployment can be limited by computational overheads and expenses, and Vibe Coding may create security risks such as hallucinations, but will be mitigated via rigorous testing and ethical guardrails.
- **Scalability and Generalisability:** Models will be designed with configurability thought of to support adaptation across various network environments and future IDS Datasets.

1.5 Structure of this Project Dissertation

This Project Dissertation is structured overall as follows to give a guide of the content:

1. **Section 1. Introduction**
2. **Section 2. Systematic Literature Review**
3. **Section 3. Methodology Used**
4. **Section 4. Implementation and Evaluation of Results**
5. **Section 5. Discussion**
6. **Section 6. Conclusions and Future Work to Incorporate Suggestions**

2 Systematic Literature Review

2.1 Introduction of the Systematic Literature Review

The constantly evolving and tense Cyber Environment, shaped by well-thought-out threats such as Advanced Persistent Threats (APTs), Zero-Day Exploits, and Adversarial Attacks, requires a removal from typical and orthodox IDS. AI-driven IDS, incorporating Deep Learning (DL), Reinforcement Learning (RL), and Hybrid Models, have emerged as promising solutions capable of adapting to evolving Attack Vectors. As advocated by Doost et al. (2025) and Ren et al. (2023), these clever architectures go beyond static capabilities, providing not only a high accuracy but also the ability to adapt dynamically. Although, to succeed in a practical deployment with business trust, technological aides such as xAI, Edge Computing, and Vibe Coding Karpathy (2024) must be integrated ethically and lawfully. This systematic review of the literature critiques all the above themes, signifying methodological strengths, limitations, and the evolving debate in the current scholarly literature.

2.2 DL for IDS

Various DL models such as Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Transformers have gained traction within IDS research, easily being able to model rich data with high dimensions and non-linearly observant patterns. They have demonstrated remarkable accuracy in benchmark Datasets like CIC-IDS2018 and NSL-KDD.

Advantages: CNNs are used to retrieve spatial hierarchies from data packets, showing great efficiency when processing static features (Doost et al. 2025). Also, Long Short-Term Memory (LSTM) networks can detect non-permanent dependencies within network flow traffic, being effective in identifying sequential attack patterns. Doost et al. (2025) also had incorporated a CNN-RF model that had resulted in the accuracy $> 97\%$ in the IDS 2018 Dataset of the Canadian Institute of Cybersecurity (CIC), exceeding that of orthodox ML classification methods. The exploration of Transformers has also occurred: Xi et al. (2024) have introduced a multiscale Transformer model called “IDS-MTran” achieving $> 99\%$ accuracy via the use of NSL-KDD and CIC-DDoS (Distributed Denial of Server) 2019 Datasets, learning contextual relationships across variable-length input sequences.

Limitations: However, despite their promise, there are limitations, such as data bias and generalisation that should reduce Class Imbalance if not done too much, as noted by Zhang et al. (2023), high computational strains, which restrict live deployments or Edge Computing environments, and poor interpretability, weakening analyst trust and confidence, as well as limiting their utility in resource-constrained or real-time environments. Contrastingly, the opposite of generalisation is overfitting, where this is another key concern due to the overparameterisation of neural networks. As mentioned earlier, Class Imbalances still remain prevalent in Cybersecurity Datasets. Additionally, DL models have hidden layers, which hinders transparency and explainability-a vital part in industries that are publicly driven by information such as healthcare and finance (Ribeiro et al. 2016b).

Conflicting Perspectives: As Doost et al. (2025) supports CNN-based ensemble methods for IDS, Charmet et al. (2022) mentions worries about how practical and reproducible they are, since they are dependant on hyperparameter tuning.

2.3 RL for IDS

Various RL models approach Intrusion Detection as an ever-changing sequential learning task-excelling in dynamic environments, adapting to ever-changing attack strategic types

via human-in-the-loop feedback-driven learning. Agents evolve and adapt through a reward system for anomaly detection via human-in-the-loop feedback-driven learning iteratively improving by using this environment-enabling suitability for live IDS.

Advantages: RL architectures such as DQNs allow autonomy of decisions, where an agent learns optimal decisions, mitigating risks. Ren et al. (2023) have proposed a model called MAFSIDS (Multi-Agent Feature Selection Intrusion Detection System), where a Deep Q-Learning multi-agent architecture is used, achieving 99.1% accuracy and 96.3% from the statistical benchmark of the F1-score. Ghubaish et al. (2024) have also used an actor-critic framework for Multiple-Access Edge Computing (MEC), showcasing RL having the ability to evolve with distributed networks, which is vital for latency-dependent.

Limitations: However, as ever, there are limitations, such as the complexity of the reward system, where RL models have sensitivity to reward design-incorrect reward design may result in suboptimal performance or even dangerous behaviour, especially in high-risk domains, higher-dimensional data convergence struggles, and ethical transparency issues of biased penalisation of specific traffic-flow types, which is unlike DL models that have specifically chosen Datasets, as well as suffering from instability when training due to the exploration-exploitation trade-off all as noted by (Gueriani et al. 2024).

Conflicting Perspectives: As Ren et al. (2023) and Ghubaish et al. (2024) highlights the advantages of distributed RL agents that can adapt to various security systems, Gueriani et al. (2024) warns that usage within IoT frameworks can result in a tendency of overfitting to local network patterns. As a result, in order to prevent this requires a need for federated RL approaches or curriculum-based training to balance adaptability and generalisability.

2.4 Hybrid Models for IDS

The objective of Hybrid Models is the combination of DL's power with RL's adaptiveness; this enables these models to be particularly specialised for detecting new threats in evolving environments.

Advantages: Strickland, Zakar et al. (2024) have proposed a DRL-GAN (Deep Reinforcement Learning- Generative Adversarial Network) model, where a GAN is used to address Class Imbalances before DRL training. Detection of rare attacks such as Bots and DDoS by generating synthetic minority samples as a data pre-processing step before training. Zhang et al. (2023) also emphasises that Hybrid models are ideal with these changing Attack Vectors, as well as enhancing attack detection rates in highly imbalanced Datasets.

Conflicting Perspectives: As some researchers such as Strickland, Zakar et al. (2024) and Ren et al. (2023) support for hybrid models as a performance-enhancing solution, Char-met et al. (2022) identifies the lack of simplicity and transparency in model design and evaluation.

Limitations: However, like always, there are limitations, such as complex model tuning, increasing computational costs, and the difficulty of explainability because there are more layers.

2.5 xAI, Bias, and Transparency

To address the lack of explanation of the feature outputs produced by these AI models, xAI models such as SHAP and LIME, and attention visualisation are used for IDS.

Advantages: A demonstration is when Houssel et al. (2024) incorporated SHAP with GPT for IDS use, where the most prominent features were variation in packets and network traffic-flow. Data Analytical Heatmaps further optimises analyst confidence and trust. Additionally, SHAP Lundberg and Lee (2017) and LIME Ribeiro et al. (2016b) provides

interpretability after testing, aiding data analysts to comprehend individual feature significance in model predictions. Attention visualisation based methods in Transformer-based IDS Xi et al. (2024) provides insights into deeper correlations, improving operator confidence.

Limitations: However, there are challenges, such as computational obstacles in live-systems, probable misrepresentation of the logic of model use as cited by Ribeiro et al. (2016b), as well the lack of statistical fairness benchmarks, such as the lack of false positives from users and/or geographical locations.

Conflicting Perspectives: Studies such as from Charmet et al. (2022) highlight that xAI is necessary for regulatory compliance but also need to be audited for sufficiency. This brings into perspective that there are a growing number of voices that xAI must be transparent and accountable rather than treated as a standalone solution.

2.6 Vibe Coding for IDS

To educate the masses and enable greater Cyber-resilience, Vibe Coding is used; this being first introduced by Andrej Karpathy (co-founder of Open AI LLM) Karpathy (2024), where a language is inputted into the LLM and code is generated, which is a form of Generative AI. This enables non-technical experts to construct IDS parts, improving accessibility, as there are lower entry barriers, and testing phase speeds.

Advantages: Vibe Coding provides accessibility for beginners to generate, test, and validate components of IDS using natural language prompts. For example, using GPT-4 to generate Python Programming Language code for anomaly detection, enabling swift prototyping-this is especially ideal for agile development frameworks.

Limitations: However, limitations emerge, such as bias and hallucinations, which are constant risks. The precision, logical flow, and security of the generation AI code must be checked against certain benchmarks as cited by (Houssel et al. 2024), where there is human-in-the-loop oversight, ensuring trust and accountability-further research is needed to address adversarial resilience and the propagation of systemic biases as there is no new significant research that calls for anything majorly different.

2.7 Edge Computing and Resource Limitations

Integrating AI-IDS specific Python Programming Language structures and techniques (explained fully later in section 4 of this Project Dissertation) to enable at the edge of networks (Edge Computing) provides advantages in delays and privacy.

Advantages: Ghubaish et al. (2024) had recently developed and deployed RL-IDS at MEC (Multi-access Edge Computing) Nodes, achieving less than 50 ms in data transmission delays, vital for IoT (Internet of Things) and healthcare infrastructure and data. Easily deployable DL models such as MobileNet and DistilBERT (from Hugging Face repository) are becoming the preferred model/s of choice/s.

Limitations: However, there are also limitations like previously discussed, such as the careful optimisation of model size, live inference remaining a challenge, limited memory, and xAI tools not being compatible with Edge Computing due to their runtime requirements (Shen et al. 2024; Song and Ma 2024; Charmet et al. 2022). Additionally, federated learning is still in the early stages, making collaborative learning difficult.

Conflicting Perspectives: While Song and Ma (2024) advocates for federated learning with attention-based networks for Edge IDS, Gueriani et al. (2024) highlights the concern of computational expenses. A hybrid Cloud-Edge architecture is recommended, where core inference locally runs on-device, and retraining occurs in centralised systems on the cloud to improve scalability and resilience.

2.8 Methodological Frameworks for IDS

To have a structured and organised plan for IDS development and deployment, researchers and developers adopt the methodological frameworks of CRISP-DM and SDLC. The components of CRISP-DM are the following:

- **Business Understanding:** Define the threats that requires attention and critical assests.
- **Data Understanding and Preparation:** Feature extraction, class balancing, log cleaning, and dimensionality reduction.
- **Modelling:** Training Hybrid DL-RL models with xAI and Vibe Coding parts integrated where appropriate.
- **Evaluation:** Use statistical benchmarks such as F1-Score, ROC (Receiver Operating Characteristic)-AUC (Area Under the Curve) curves, and explainability for validation, as well as all of this with Stratified K-fold cross validation.
- **Deployment:** Enabling deploying models on Cloud/Edge Computing architecture and infrastructure by using certain Python Programming Language structure and techniques.
- **Monitoring:** Use feedback loops for retraining and rewards for RL.

This goes hand-in-hand with the SDLC methodological framework by focusing on organisation and planning, sequential development, and some form of recording. Agile methodological frameworks (an emphasis on collaboration, continuous improvement, and fast responsiveness to change) are designated for Vibe Coding pipelines as it's essential for quick prototyping.

Advantages and Disadvantages: These methodological frameworks establish clarity and consistency to development, but have challenges in AI-Driven IDS contexts due to models not being deterministic.

2.9 Transparency, Fairness, and Accountability for AI within IDS

As AI systems become increasing autonomous, making vital security decisions, accountability is extremely important. Ribeiro et al. (2016b) support in-depth auditing, human-in-the-feedback-loop review, and fairness evaluations. Good AI-IDS should:

- Have bias assessments such as True Positive Rate imbalance score, etc.
- Record decisions for audit purposes.
- Provide xAI reasons for transparency purposes.
- Enable human overriding capabilities and ethical oversight and intervention (if deemed necessary).

Limitations: Many publications fail to meet these requirements, potentially facing non-compliance with existing regulations such as GDPR and emerging ones such as the EU AI Act. Researchers must adapt to balance these performance goals and objectives with governance rules and regulations-a model with 99% accuracy may still be discriminatory or opaque.

Future Pathways: Zhang et al. (2023) addresses the need for realistic data and diversity, highlighting that lots of existing ones lack context and geographical representativeness. Insights, fairness and accountability should be integrated into the design of AI-Driven IDS from the get-go.

2.10 Summary of the Literature Review

In recent years, the field of AI-driven IDS has witnessed rapid and transformative progress. Successive advances in DL, RL, and hybrid model architectures have achieved successful detection rates that were not seen previously. This progress has further been boosted by the incorporation of xAI, and Edge Computing compatible methods, enabling live monitoring with better robustness and efficiency on Edge Devices.

However, while there has been remarkable progress, significant challenges remain. Important issues such as model interpretability, explainability, live deployability, bias, and adherence to ethical standards pose ongoing hurdles. To address these, it is essential that future research rigorously incorporates methodological frameworks such as CRISP-DM and SDLC. These frameworks ensure that AI-IDS development is not only driven by benchmark metrics but equally prioritises transparency, fairness, and accountability. Once these principles are embraced, AI-IDS solutions gain the full trust and confidence of users, ultimately enabling their successful deployment in complex, real-world environments.

This project aims to resolve these challenges by thoroughly investigating into the design, implementation, and evaluation of a hybrid DL-RL intrusion detection model. xAI, Vibe Coding, and Edge Computing compatible methods by using certain Python Programming Language structures and techniques (explained fully later in section 4 of this Project Dissertation) are integrated (as suggestions mainly), further enhancing this approach. There are clear and defined objectives, addressing the specific limitations in current AI-IDS research, guiding the selection of relevant Datasets, detailed data pre-processing steps, and the justification of chosen models — including a critical analysis of their limitations.

Moreover, the research methodological frameworks are structured to maximise the validity and reliability of the discoveries, with ethical considerations cautiously incorporated throughout the process. Time management and project planning are systematically laid out via a Gantt chart (in the Project Proposal) to ensure disciplined progress. The literature review gives a meticulous contextual foundation, delving deeper on the research motivations and providing clarity of the contributions of the rest of the dissertation.

After thorough thought, the final model proposed for deployment is a **hybrid DL-RL architecture augmented with xAI (suggestions), Vibe Coding (suggestions), and Edge Computing compatible components by using certain Python Programming Language structures and techniques (explained fully later in section 4 of this Project Dissertation in terms of Edge Computing Compatibility)**. This particular combination aims to deliver excellent detection capabilities, better interpretability, and deployability, recording significant progress of trustworthy AI-IDS.

3 Methodology Used

This section is organised into four interconnected pillars 1. Dataset Selection and Characterisation, 2. Data Pre-processing and Feature Engineering, 3. Model Development and Hyperparameter Optimisation, and 4. Evaluation and Deployment Framework. Each pillar is examined in depth, with explicit pros, cons, and conflicting perspectives from the latest literature.

3.1 Datasets Used

All of this is only possible of course with ensuring that the data obtained is of superb quality. Remember, this is one of the key foundations and fundamentals of any robust IDS to get right

in order to ensure that the performance is optimal, where this lies in the quality, diversity, and representativeness of the Datasets used to train and evaluate detection models.

Three Datasets are used; carefully they are tailored for excellent training and evaluation of the proposed AI-Driven IDS, ensuring a comprehensive and realistic assessment of the AI-driven IDS proposed in this study. Each of these Datasets have specific strengths and limitations, offering complementary viewpoints on network traffic, attack types, and feature structures. By integrating insights from all three, the proposed system is evaluated in varied conditions that mirror legacy, modern, and volumetric cyber threats:

- **NSL-KDD:** An iterative update and improvement of the classical and original KDD'99 Dataset, NSL-KDD resolves the redundancy and imbalance issues, removing duplicates and balances the number of samples across different difficulty levels, thus allowing fairer training and testing without bias from repeated patterns (Tavallae et al. 2009a). It contains 41 features that encompasses continuous and categorical variables as well as labelled network flow traffic data with four main attack categories (features): DoS, Probe (information gathering of a targeted network for either preparation of a Cyber-attack or exploiting vulnerabilities) Editors of Imperva (2025), R2L (Remote-to-Local) attacks (when Cybercriminal/s send network packets to victim/s computer/s/system/s to learn their behaviour and gain access that the victim/s would have) *Effective Analysis on Remote to User (R2L) Attacks Using Random Forest Algorithm.* (2014), and U2R (User-to-Root) attacks (when Cybercriminal/s have admin access to victim/s computer/s/system/s that is non-privileged) *what is u2r attack* (2025). Commonly used for benchmarking IDS models, supporting traditional ML methods well and is ideal for early-stage model testing, but limited in scope as modern prevalent threats, such as ransomware, are not addressed (Python Developer 2025), as well as evolving malware, encrypted traffic, and lateral movement. Furthermore, Class Imbalances are especially prevalent in the U2R and R2L categories, which have significantly fewer instances, thereby posing a challenge for accurate classification-such imbalance can result in biased models that fail to generalise beyond the dominant classes (Moustafa et al. 2015).

Nonetheless, this Dataset is still valuable as a baseline due it being simple, readily available, and compatible with classical ML algorithms. It is especially of use in benchmarking new models before moving on to more complex Datasets.

- **CIC-IDS2018:** Published by the Canadian Institute for Cybersecurity, this Dataset includes benign and malicious network traffic flow including botnets, brute force, DDoS, DoS, infiltration, and web attacks. As a result, modern traffic patterns are introduced. This Dataset contains practical scenarios with the latest Attack Vectors, recent attack scenarios created in a realistic environment, and packets suitable for DL algorithms for detection systems.

This Dataset contains over 80 NetFlow and packed header analysis features, including Statistical and behavioural indicators such as flow duration, packet size variance, and TCP flag counts. As a result, greater context is provided for DL models, especially those reliant on temporal and behavioural patterns. Moreover, the data are timestamped and organised by day, offering the possibility of temporal and sequential analysis.

However, this largest Dataset is computationally expensive for processing and training, particularly for DL models, and is likely to create Class Imbalances (solarmainframe 2018), since some attacks-such as DDoS and Brute Force-are overly represented, while others are sparsely included. An imbalance occurs as a result of this, which must be carefully addressed during preprocessing to avoid biased model predictions.

There is also the issue about the realism of lab-generated traffic. As a result, models trained solely on this Dataset may struggle when deployed in production environments with more heterogeneous and dynamic traffic patterns (Shiravi et al. 2012).

- **CIC-DDoS2019:** Specific to DDoS attack types only, thus is the most specialised out of the three Datasets. Contains multiple DDoS types (over 50 different types of Attack Vectors) such as SYN Flood (restrict or remove all legitimate network traffic flow to a server by sending requests, overwhelming the server, causing it to respond slowly or not at all) *What is a SYN flood attack?* (2025), UDP (User Datagram Protocol) Flood *UDP flood attack* (2025) (time-reliant packets, where communication with the server is sped up before a connection is officially formed, overwhelming the server and firewall, preventing actual network traffic from being received) *What is UDP?* (2025), HTTP Flood (overwhelming a targeted server with lots of website links, effectively reducing or ceasing its operation/s) *HTTP flood attack* (2025), and also others. This Dataset enables detailed evaluation of IDS performance against high-volume DDoS threats (Rakib Hossain Sajib 2019; FieryBanana101 2019) live or near-live conditions. **Note:** this particular Dataset from FIERYBANANA 101 on Kaggle only has two labels: “BENIGN” and “Syn” (FieryBanana101 2019). Despite the label “Syn” which specifically identifies SYN flood attacks, inspection revealed that this label had been used to represent all forms of malicious activity in the Dataset, despite the original CIC-DDoS2019 that contains more than 50 different types of DDoS attacks. In order to avoid misrepresentation, this label is recoded to ATTACK as a better representation of the true binary nature of the classification task. This ensured consistency with the binary labelling scheme adopted across NSL-KDD and CIC-IDS2018, where all malicious traffic was labelled as 1 and all benign traffic as 0.

However, its specialised focus reduces generalisability of training IDS models, which means that models trained solely on this data may overfit to volumetric threats while missing more covert or stealthy intrusions (Ullah et al. 2020).

NOTE: Both of the original CIC-Datasets are extremely large, so authors who have revised down and filtered the original Datasets who had encountered similar issues to this Project acquiring the relevant Datasets have kindly uploaded these onto the Kaggle Repository. Here are the following links to these publicly available Datasets: Ekky Kharismadhany 2018 Ekky Kharismadhany 2018 that were used as alternatives.

These Datasets provides detailed and varied foundations for evaluating performance metrics for DL, RL, and hybrid DL-RL IDS models. All of these Datasets are available on the Kaggle repository as CSV (Comma Separated Variable files), where descriptions can also be found along with them.

3.1.1 Justification of multiple Datasets Used

Since there are limitations of using a single Dataset, this study adopts a multi-Dataset strategy. NSL-KDD is used to benchmark performance on a Dataset with distinct class separation. CIC-IDS2018 provides an up-to-date and diversified attack environment with flow-based features ideal for DL architectures. CIC-DDoS2019 tests the scalability and resilience of the model under high-volume adversarial conditions.

By combining these Datasets, the IDS can be tested across a broader spectrum of network behaviours and attack surfaces. This approach enables a suitable amount of generalisabil-

ity and forms realistic scenarios where the system may encounter varied forms of network anomalies.

3.1.2 Advantages of Multiple Datasets Used

The advantages of multiple Datasets being used are the following:

- Provides in-depth evaluation of classical ML, DL, and RL-based IDS models.
- Covers previous, present, and large-volume Attack Vectors for a fair experimentation.
- Allows multiple Datasets to be cross-validated to examine generalisability and reduce Dataset-specific overfitting.
- All the Datasets are on a public data repository, enabling reproducibility and transparency of results.

3.1.3 Disadvantages of Multiple Datasets Used

Taking into consideration, the disadvantages of multiple Datasets being used are the following:

- Requires thorough preprocessing to coordinate and systemise different feature sets, scales, and labels.
- Balancing rare attack types remains challenging, notably in NSL-KDD and CIC-IDS2018.
- The Datasets are also varied possibly introduces Domain Shift (source training data characteristics are significantly different from the test label/s) (Tran et al. 2024).

3.1.4 Opposing Academic Perspectives

Some researchers voice their support for combining multiple Datasets to improve generalisability, while others are against due to feature distribution shifts that will confuse the models during the training and testing phases. Combining multiple Datasets without robust and rigorous domain adaptation techniques may result in lower performance on unseen data (Ferrag et al. 2022). Addressing these concerns are the possible incorporations of the techniques of Domain Adversarial Neural Networks (DANN) or feature alignment layers, which help models learn domain-invariant representations and better transfer knowledge across varied environments.

3.2 Data Pre-processing Steps Used

Now, to ensure that performance metrics are the best that they can be, effective pre-processing is vital for constructing rigorous and robust and generalisable AI-Driven IDS. It is not a simple preparation step, but a foundational phase that dictates how well the downstream models can learn patterns from the data. The following pre-processing techniques are implemented, based on best practices recommended by Pedregosa et al. (2011) and more recent improvements in scikit-learn, given the diversity of the selected Datasets, the pre-processing strategy must ensure feature consistency, class balance, noise reduction, and computational efficiency. This section goes in-depth of the pre-processing methods implemented, supported by academic rationale, practical implications, and potential trade-offs as follows:

- **Missing Value Imputation:** One of the main and initial pre-processing steps. The CIC-IDS2018 and CIC-DDoS2019 Datasets, respectively, are large, typically containing corrupted entries or null values, especially in the columns called “Flow Byte/s” and “Flow Packet/s”. These values can occur due to incomplete simulated packets or zero-flows. Instead of deleting rows completely, reducing data representation especially rare attack types, a thought-out strategy was implemented. Columns with numerical values with missing values were imputed using median values, which are more robust against outliers than the mean, matching with Pedregosa et al. (2011) practices. Categorical column items used the mode as the imputation.

Columns that are thought to have no impact on the final classification are removed, such as flow ID (Ullah et al. 2020). This is due to irrelevant features that affect pattern recognition and computational overload (Ullah et al. 2020).

- **Feature Encoding:** Feature Encoding is used in this instance to standardise all the target variables across the three Datasets mentioned earlier (HOPSWORKS Editors 2025). To ensure consistency, Binary Encoding entails the labels, where the number 1 is assigned to malicious malign attack activity and the number 0 is assigned to benign normal activity, thus this is a practical objective of many real-world IDS systems. The generalisation of models is improved across heterogeneous sources, aligning with operational goals in real-world security systems, thus being practical (Abdulhammed et al. 2019) (Ferrag et al. 2022).

Even with One-Hot Encoding applied to input features with few unique values such as “protocol_type” and “flag” in NSL-KDD, the main encoding task was to create a robust and interpretable target variable. The use of binary label encoding supports improved scalability and model portability across environments with evolving threat types (Shafiq et al. 2020) (Ullah et al. 2020). As a result, class fragmentation is reduced, as it enables Datasets to be combined into a single training corpus with consistent semantic meaning in the target variable, enabling the model to learn a unified representation of what constitutes an intrusion versus normal behaviour, simplifies evaluation metrics, and avoids unnecessary complexity in label interpretation, thus improving scalability and the portability of models across varied environments with evolving threat types (Shafiq et al. 2020) (Ullah et al. 2020).

- **Feature Normalisation and Scaling:** Feature Scaling is crucial for IDS, as there is a range of numerical values, where features such as “Total Fwd Bytes”, “Flow Duration”, and/or “Packet Length Variance” can span over large orders of magnitude. Based on these requirements, two scaling methods were used as follows: StandardScaler (Z-score) and MinMaxScaler are applied depending on model sensitivity. For example, neural networks and LSTMs use MinMax scaling (between 0 and 1) for the classification of outputs, where Standardisation possibly results in exploding or vanishing gradients, whereas tree-based models are probabilistic and thus use standardisation (Goodfellow et al. 2016a). Min-max scaling maintains bounded feature input, enabling greater chances of stability in convergence during training (Goodfellow et al. 2016a).

Furthermore, a log transformation is applied to skewed features, such as the columns “Flow Bytes/s” and “Packet Length Mean” in the CIC-IDS2018 and CIC-DDoS2019 Datasets, respectively, reducing their kurtosis, as larger tailed distributions lead to higher data variance and greater computational overloads, resulting in slower learning.

- **Resolving Class Imbalances:** SMOTE is for balancing Datasets such as CIC-IDS2018, but risk mitigations are validated by synthetic means using cross-validation (CV)

(Chawla et al. 2002). However, the issues are that it can create very simple points that does not capture complex decision boundaries, as well as introduce noise where there is a high concentration of minority classes, reducing the generalisability of the model/s (Batista et al. 2004). Instead SMOTE-ENN (Edited Nearest Neighbour) was used, which is a combination of two techniques. Overlapping is cleaned using ENN that is an addition to SMOTE, improving class distinction (Batista et al. 2004).

- **Dimensionality Reduction:** Feature selection is vital for improving model interpretability, overfitting reduction (improving generalisability), and improving computational efficiency, especially with the CIC-IDS2018 and CIC-DDoS2019 Datasets, which contain over 80 flow-level features. The specific techniques are: **Correlation-based filtering**, which is an initial step to identify and address multicollinearity among features. Both Pearson (measures linear relationships of variables within a continuous interval or data consisting of a ratio, assuming that the data is linear and normally distributed, but is sensitive to outliers, skewing the value of correlation, affecting feature true contributive representation. Covariance and standard deviation of raw data, with Correlation Coefficient ranges of $[-1, 1]$ from negative, zero, or positive Correlation Coefficient values) GeeksforGeeks Editors (2025) and Spearman correlation-based filtering (measures the monotonicity, where any variables are consistently moving in one direction that may not be linear that is compatible with ordered data, data that is ranked, interval data, or data consisting of ratios. No normalisation or linearisation is required, working well with data that has no parameters, being resistant towards outliers as rank is used, calculating the difference in ranks, and has ranges of $[-1, 1]$ from negative, zero, or positive correlation monotonicity) GeeksforGeeks Editors (2025) were calculated, and highly correlated features (Correlation Coefficient > 0.95 (the reason being that correlation is not necessarily causation, with highly correlated figures making it increasingly difficult to analyse the unique contributions of each figure)) IntellectusConsulting Editors (2025) were identified and specifically removed, avoiding redundant learning and inflated model variance, where multicollinearity was avoided Guyon and Elisseeff (2003) Kuhn et al. (2013). As a result, model stability and interpretability improved without significant information loss, as well as the efficiency was improved, as less redundant features are used, reducing computational expenses, especially dealing with these three large Datasets combined. **Recursive Feature Elimination with Cross-Validation (RFECV)** is then applied to select the subset of features that provides the most amount of information. Recursively, it removes features of the least importance and selects the combination that yields the highest cross-validation performance (Kohavi 1995) (Guyon and Elisseeff 2003) (Chandrashekar et al. 2014). Despite being computationally expensive, RFECV can guarantee at the empirical level. Finally, **Principal Component Analysis** is used mainly for Exploratory Data Analysis (EDA) and visualisation. PCA reduces dimensionality by transforming features into principal orthogonal components that retain most of the characteristics and variance of the Dataset. However, it is an unsupervised technique and the features are generally components of linear combinations, which reduces interpretability. Thus, caution is taken when PCA is used and only in conjunction with interpretable models or visualisation steps (Jolliffe et al. 2016) (Samek et al. 2017). To ensure xAI (suggestions), PCA is not used in the final models deployed themselves (Jolliffe et al. 2016) (Samek et al. 2017).
- **Noise Detection and Outlier Dealing:** Outliers can be genuine attack patterns or noise that is just generated. In order to prevent potentially useful data from being

removed, this approach is used: **Isolation Forest**: Flags extreme outliers within continuous variables. Can deal with high dimensional data and can be unsupervised (Liu et al. 2008). (**NOTE: This was taken only as a confirmation step, taking extra caution**)

As some researchers support the removal of more outliers, this can be counterproductive in IDS, where many legitimate attacks are statistical outliers by design. Therefore, the concentration was only on cleaning corrupted entries, not on legitimate extremes in this particular instance.

Data Partitioning and Loss Prevention: All Datasets used the split training, validation, and test sets with a split of 80-10-10 ratio. Splitting occurred before pre-processing, ensuring that the removal of synthetic samples and/or outliers did not affect the test set realness, as these were only removed from the training set. Extra precaution was taken to prevent data loss, such as time-stamped sessions across splits (basically leaving the time feature column in) (Shafiq et al. 2020).

Python libraries are used within Google Colab IDE for the above steps such as Scikit-learn, etc (later discussed in this Project Dissertation), ensuring reproducibility, scalability, and alignment with current AI development standards.

The ones that are not are explained later within this Project Dissertation in greater detail.

3.3 Models Used and their Limitations Explained In More Detail

Once the data has been sifted through, in order to acquire intelligent, adaptive detection in network security, multiple model architectures need to be explored. A combination of classical and current ML models, including DL and RL strategies, evaluating the varied effectiveness of various IDS architectural strategies. In line with current Cybersecurity research, a hybrid DL-RL model is also explored to leverage the strengths of both models. Each model is selected based on its tailored suitability, empirical performance in IDS, and ability to handle different feature types and data structures. The performance of these models is critically evaluated, along with their limitations regarding interpretability, scalability, training complexity, and live deployment:

DL models are especially suited to IDS as they can learn intricate, non-linear representations of data with many dimensions. In this Dissertation, both CNNs and LSTMs are utilised.

- **CNNs**: Excellent for recognising spatial hierarchies of packet data but not for sequential data (Doost et al. 2025). They are effective at capturing local spatial patterns in structured data. IDS features can be arranged as 1D vectors or matrices that resemble image-like grids, allowing CNNs to detect local feature interactions. Usually CNN models have architectures consisting of three convolutional layers (layers that recognise patterns with architecture mimicking Neuronal patterns of the Human Brain and is inspired by the organisation of the Visual Cortex, converting all the pixels in the field that has been received into a single value that is a vector, essentially two or more interconnected sources of information that is an operation that changes a function into something different) Geeksforgeeks Editors (2025m) with increasing filter sizes (the area mapped to on an image or function (features considered)) Pandey Swarnima (2020) (32 (optionally), 64, and 128, etc), each followed by ReLU (Rectified Linear Unit) activation function (a simple piecewise function that has a linear property outputting explicitly the input if positive; otherwise an output of value zero) Geeksforgeeks Editors (2025h) and max-pooling layers (typically a reduction of spatial layers in terms

of height and width of the mapping of the input features (usually images) while keeping critical information by placing a 2D filter over individual channels of feature maps, summarising the features within the bounds of the filter that has covered) Geeksforgeeks Editors (2025b). Dropout Regularisation (a technique within the certain hidden layers that ignores specific Nodes, effectively getting rid of them by zeroing them out, where the appearance going forward is changed, reducing overfitting due to particular instances of co-adaptability of certain network layers as a result of prior layers, improving robustness by increasing generalisability; this technique is being applied to every single hidden layer only, and as a stand alone layer within the hidden layers) (usually 0.3) Geeksforgeeks Editors (2025d) Marimuthu Parthiban (2024) Wu et al. (2015) and batch normalisation (a technique aimed at improving the training process. The aim being to resolve the issue of Internal Covariant, where the distribution of input layers changes due to the change of the parameter of the previous layer (Geeksforgeeks Editors 2025f). The activations distributions (output layers) alters during the training phase due to network parameters updates. As a result, every layer has to adapt at a continuous rate to the inputs and their changing distributions received from the previous layers. Consequently, this can slow down the training phase and increase the difficulty of model convergence) Geeksforgeeks Editors (2025n). These are also used to prevent overfitting and stabilise learning. They are computationally efficient, and they can be deployed in Edge Environments, making them attractive for resource-constrained IDS deployments. However, they lack temporal memory, limiting their ability to model sequential patterns. In addition to this, after the convolutional and pooling stages, a Flatten layer was applied to transform the multi-dimensional feature maps into a one-dimensional vector. This vector then goes through dense (fully connected) layers, which perform high-level reasoning by learning weighted combinations of all extracted features. Dense layers act as the final integration stage, combining the hierarchical features detected by the convolutional layers to make a classification decision. Further enhancing model robustness, were the implementation of L1 and L2 regularisation to dense layers. L1 regularisation (also known as Lasso) promotes sparsity by placing a penalty on the absolute weight magnitudes, performing feature selection in the new learned representation (Geeksforgeeks Editors 2025g). L2 regularisation (also known as Ridge) places penalties on the squared magnitude of the weights, promoting smaller weights to reduce complexity of the model, reducing overfitting and improving generalisability (Geeksforgeeks Editors 2025g). A combination of L1 and L2 regularisation (also known as Elastic Net) was used to balance sparsity and stability Geeksforgeeks Editors (2025g) (note that the very final output activation layer is a sigmoid, which is a S-shaped function that introduces non-linearity enabling complex decision boundaries for complex problem for Binary Classification) (Geeksforgeeks Editors 2025i). Since this was a Binary Classification task, the Binary Cross-Entropy loss function was used. It explicitly measures the dissimilarity between the predicted probability distribution and the actual class labels (Saxena Shipra 2024). Hyperparameter Tuning was also carried out using GridSearch CV mimicker (a Tuner), which completely searches over specified parameter grids such as filter sizes, dropout rates, learning rates, batch sizes, etc, identifying the optimal configuration for model performance (mimicked by selecting a random subset up to a specified maximum number of trials, selecting based on validation accuracy of the best one, which is much faster and less computationally expensive). Despite being computationally expensive, this approach guarantees in-depth exploration of the hyperparameter space, which is vital for IDS where even the tiniest architectural alterations can significantly impact generalisation to novel attacks. In

short, CNNs were chosen for their computational efficiency and suitability for edge deployments in resource-constrained environments. On the other hand, they cannot retain temporary dependencies, rendering them unable for sequential network traffic flow analysis, hence other models are considered, which are below (Yin et al. 2017). **(NOTE: As the CNN processes tabular features without temporal memory, shuffling the Datasets since they are combined does not affect performance.)**

- **LSTMs:** Excellent for time-dependent sequential data such as network traffic flows, but are computationally costly and sensitive towards hyper-parameters (Zhang et al. 2023). They are a form of Recurrent Neural Networks (RNNs) (type of neural network that enables previous outputs to be used as inputs while being hidden (not visible to the user either as an input or output)) Amidi, Afshine and Amidi, Shervine (2025) Geeksforgeeks Editors (2025k) designed to capture temporal dependencies in time-series data. This property makes this model ideal for analysing packet sequences, connection durations, and time-based anomalies in network traffic. In this implementation, the LSTM model consists of two LSTM layers with 32, 64, and 128 units (number of Nodes) respectively, followed by fully connected layers and a sigmoid output node (a S-shaped function that introduces non-linearity enabling complex decision boundaries for complex problems) Vishwakarma Swapnil (2024) for binary classification. They can show more than CNNs for time-dependent behaviour but require more training time and are sensitive to vanishing gradients (an issue that occurs during Backpropagation Geeksforgeeks Editors (2025a) where the steepness of the slopes of the activation functions become eventually smaller through previous layers of a neural network, especially with many-layered deep networks, hampering the training of the model as the updates of the weights become minute, prolonging the training duration, or completely halting the training phase) Geeksforgeeks Editors (2025l) and overfitting on noisy or imbalanced Datasets. Furthermore, the model's sequential computation limits parallelism and live performance in systems that require a lot of information. **(NOTE: The LSTM model in this case uses a similar layout and format as the CNN model for convenience, which are using the a dropout rate from 0.3 Geeksforgeeks Editors (2025d) Marimuthu Parthiban (2024) Wu et al. (2015), a dense layer followed by a RELU activation function, Binary Cross-Entropy loss function, and Hyperparameter Tuning using GridSearchCV mimicker (a Tuner)).**

CNN and LSTM models rely on well defined features. Despite feature engineering being partly reduced by DL architectures of the hidden layers, standardisation, and encoding, these remain vital for the performance of these models. Moreover, these models act as hidden classifiers, making them less suitable in environments where explainability and compliance such as GDPR and the EU AI Act are required.

RL provides a dynamic and policy-driven decision-making model. RL interacts with an environment and learns optimal actions by maximising cumulative reward, unlike Supervised Learning, which inactively learns from labelled data. With regard to IDS, systems are enabled to adaptively respond to traffic patterns, refine detection strategies live, and even take autonomous mitigation steps such as minimising complex alerts to reduce overwhelming or increase the number of alerts if seen fit to do so.

- **DQNs :** DQN model that adapts to responses via RL, enabling dynamic threat recognition. However, there are instabilities in training and the challenge of the exploration-exploitation trade-off (Ren et al. 2023) (finding a balance between exploring the given environment by attempting random executions to discover more information about the environment and exploiting understood information to maximise the reward). The RL

environment was designed such that each state corresponds to a network traffic flow (represented by preprocessed feature vectors), and the agent takes a binary action: 1 to classify it as an attack, and 0 as benign. The reward function assigns a positive reward for correct classifications and a penalty for misclassifications, guiding the agent to improve detection accuracy over time.

Replay with experience (a well defined-size buffer containing the most recent transitions (learnings) by the policy (algorithm (a series of well-defined instructions)) , significantly optimising the sample efficiency if the algorithm by facilitating the data for training to be used multiple times) is used for training stabilisation by removing pairings of correlated samples, improving convergence Fedus, W., Ramachandran, P., Agarwal, R., Bengio, Y., Larochelle, H., Rowland, M. and Dabney, W. (2020) Liu, R. and Zou, J. (2017). 100,000 transitions replay buffer sizes are used and training batches were randomly sampled to reduce variance. The Epsilon-greedy policy (a RL agent is not told at all what to do, but must use trial and error to learn which action yields the maximum reward, where there is uncertainty of deciding which path to choose to ensure maximum rewards occurs. A probability based action-value function estimate is used since the value of selecting for an action-value function is unknown to the agent, so an average of the sample is taken to estimate the value of an action being taken. A balance must be achieved between being greedy as to keep with the same action-value estimates, exploiting them to get the most reward, and having a high Epsilon (neighbourhood (width) of a function), which means a high probability of choosing to explore. However, greedy does not guarantee maximal reward, leading to sub-optimal performance, and having a high Epsilon means exploring too much, meaning computational expenses, so is inefficient, hence why a balance is essential) Geeksforgeeks Editors (2025c) Geeksforgeeks Editors (2023) Fedus, W., Ramachandran, P., Agarwal, R., Bengio, Y., Larochelle, H., Rowland, M. and Dabney, W. (2020) Liu, R. and Zou, J. (2017) regulated exploration starting with high randomness ($\epsilon = 1$) and gradually decaying towards exploitation. Target networks are updated sequentially at a certain number of steps, improving stability (Mnih et al. 2015).

However, single RL models by themselves struggle within the IDS context due to a lack of reward signals, lack of actual live truth, and the non-stationary nature of network traffic. Additionally, training RL agents is computationally expensive, and poor hyperparameter tuning can result in poor performance or dangerous behaviours.

- **DL-RL Architectures:** A combination of LSTM with CNN or DQN with RL improves adaptability and detection, overcoming the limitations of pure DL or RL approaches. This architectural design is modular in nature, where a DL model such as a CNN or LSTM is serving as a Feature Extractor (reducing the dimensionality of complexity with the intention to improve the performance and/or efficiency of various ML algorithms) or Encoder (features converted into a different format for either data compatibility (this case for non-numerical values into numerical ones) and/or improving model performance, such as adjusting the size of inputs to a specific size), and its output is passed into a RL agent such as a DQN that makes classification decisions based on the Learned State that is presented. The DL component processes Raw or Encoded Features, transforming them into a space for Latent Embedding (data that is compressed, preserving only the most important features that shows the input data's fundamental structure). This is used as a Vector State for the RL agent, which learns an Optimal Classification Policy through trial and error. This structure allows the system to benefit from DL model's powerful Feature Extraction and RL model's ad-

aptability and dynamics. In addition to this, Categorical Features were included in the input State Space after relevant One-Hot Encoding, enabling the model to learn Policy Behaviours that are conditional based upon these Categorical Features.

This model layout showcased improved robustness to model drift and previously unseen patterns, as it could adjust its decision-making policy over time. Furthermore, outperformance occurs in dynamic environments compared to stand-alone models. However, these models are computationally intensive, the dual-stage architecture requires separate optimisation strategies, and convergence is sensitive to hyperparameter alignment between DL and RL components, and interpretability isn't as good as the previous two models, as the architecture becomes increasingly complex (Strickland, Zakar et al. 2024).

When deploying these types of models, they also require more memory and computational resources, resulting in less practicality for Edge-based IDS (IDS that is compatible with Edge Devices by using certain Python Programming Language structures and techniques (explained fully later in section 4 of this Project Dissertation)) unless model compression techniques such as quantisation (reducing the executional space of a model by compressing a digital signal so that it takes up less space, computational resources, ultimately reducing processing times while minimally impacting and reducing the accuracy) CLOUDFLARE Editors (2025) or knowledge distillation (transfer the information gained from a larger complex model to a smaller compact model) Bergman Dave (2025a) are applied.

In summary, classical DL models such as CNNs and LSTMs are highly effective at recognising static or temporal patterns in known Datasets, particularly when trained on well-balanced, well-structured data. However, adaptability remains of concern without retraining and offers limited interpretability. RL, contrastingly, is adaptable and has higher levels of autonomy but is harder to train, particularly in noisy, sparse, or imbalanced environments such as network traffic. The hybrid DL-RL approach offers a compelling middle ground and resolution, but introduces complexity and training challenges.

Stratified K-fold CV, benchmark metrics, and model explainability are also accounted for (Lundberg and Lee 2017; Ribeiro et al. 2016b) and are explained later on in this Project Dissertation.

3.4 CRISP-DM Flowchart, CRISP-DM, and SDLC Methodological Frameworks and Methodology Explained In More Detail

All of section 3 is only possible with initial research and a main methodological framework (CRISP-DM in this case, which is already explained-this is just a summary in a flowchart format) as to justify all of the above, which is demonstrated in the following flowchart:

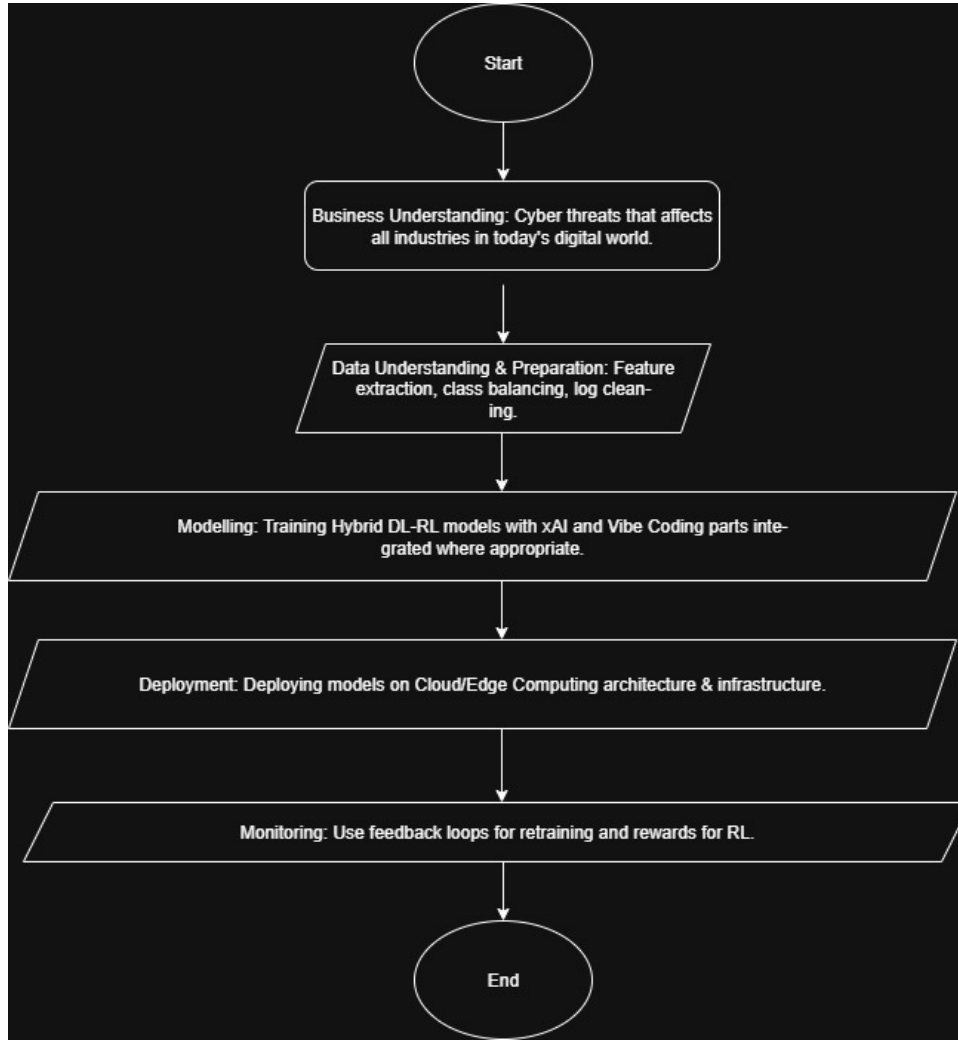


Figure 1: A flowchart showing how CRISP-DM has been implemented for the Methodology Used above.

This was also supplemented by principles from the SDLC methodological framework as well. Using both these frameworks offers an organised Data Science/ Analysis workflow and a robust Data Engineering methodology to develop, evaluate, and simulate deployment of an AI-driven IDS. As CRISP-DM aided the exploratory, iterative, and model-building phases Shearer (2000) Wirth et al. (2000), the SDLC principles were used to ensure secure, maintainable, and ethically designed software components for potential real-world deployment, thus making it practical.

3.4.1 Business Understanding

Understanding business needs is a fundamental component of this methodological framework. The main objective was to create a robust AI-Driven IDS with detection of known and previously unknown intrusions by using DL and RL. In the context of business-the dynamic and constantly evolving Cyberattacks, emphasising IDS that employs dynamic mechanisms (Ferrag et al. 2022). From a SDLC perspective, these are the requirements and analysis stage, where the user needs to ensure all the success metrics such as detection rate, false positive rate, adaptability, and explainability are explicitly defined, as well as live low-latency for live inference and ensuring that this software can be deployed on edge devices.

3.4.2 Data Understanding

Understanding the data is also vital to make use of them; this is demonstrated by selecting three Datasets based on the features and structure of the Datasets NSL-KDD Tavallae et al. (2009b), CIC-IDS2018, and CIC-DDoS2019 (Sharafaldin et al. 2018). All three contain heterogeneous features and non-uniform distributions of attack types. Each Dataset was analysed for Class Imbalance, feature skewness, missing values, and relevance to modern attack patterns. From an SDLC viewpoint, this aligns with feasibility analysis and technical specification, where the system's foundational inputs are evaluated for integrity, coverage, and alignment with real-world network conditions. This guarantees that the data used as input for the models later on were relevant, varied in attack types, and reflective of the current threat environment.

3.4.3 Data Preparation

A range of pre-processing tasks were encompassed within this step. Categorical features were all encoded in one-hot way, converted to numerical values using DL layers. Missing values were imputed using Mode or Median strategies appropriately (previously explained in more detail in Section 3.2). Highly correlated features (Correlation Coefficient > 0.95) were removed using Pearson and Spearman correlation analyses (Guyon and Elisseeff 2003), and RFECV was used to identify the most informative features (previously explained in more detail in Section 3.2) (Chandrashekar et al. 2014). PCA was applied for exploratory dimensionality reduction, but was not used in the final modelling, preserving interpretability (Jolliffe et al. 2016) (Samek et al. 2017). Label unification ensured that all three Datasets shared a binary label structure (0 = benign, 1 = attack (malign)), enabling unified training.

From an SDLC perspective, this represents the design and prototyping stage where the data pipeline is engineered for repeatability, scalability, and traceability. Version control for Datasets and preprocessing scripts was managed using Git, up-to-date Development Security Operations (DevSecOps) principles for reproducible research and secure software development.

3.4.4 Modelling

The particular stage of modelling involved the design and training of multiple algorithms, including CNNs, LSTMs, DQNs, and a hybrid DL-RL architecture. All models were meticulously evaluated using precision, recall, ROC-AUC, and F1-Scores. CNNs gathered local patterns in the feature space, while LSTMs were leveraged to sequentially detect attacks. The hybrid model passed Latent Embeddings from the DL component to an RL agent to learn adaptive classification policies over time (Strickland, Sharma et al. 2024).

From the SDLC perspective, this corresponds to the implementation and unit testing stage. All models were tested separately and individually to see how they performed. Hyperparameter tuning using grid search and performance monitoring was also used, ensuring both quantitative and qualitative evaluation of dynamical learning.

3.4.5 Evaluation

The evaluation phase in CRISP-DM tested each model against unseen test data drawn from all three Datasets. Key performance metrics included accuracy, precision, recall, F1-score, and AUC-ROC curves.

This phase aligns with the verification and validation stages of the SDLC. Verification was to ensure that the system met its design specifications, while validation was to confirm

that it achieves the desired practical performance. Explainability tools were integrated at this stage to satisfy ethical design principles and prepare for potential regulatory compliance, especially under rules such as GDPR and the EU AI Act.

3.4.6 Deployment Potential

As mentioned before about enabled Edge Computing via the use of specific ways of Python Programming Language being used, these include:

- **Combining Datasets:** The combination of Datasets reduced compute time and latency, meaning that easy Edge deployment was potentially possible.
- **One-Hot Encoding:** A standardisation for all the Datasets for uniformity, where less computational power is required to work out how the Datasets work, enabling easy Edge deployment.
- **Concatenation and Alignment of missing columns:** Combining and filling in missing columns reduces less variance of data and compute power for figuring out what to do with the missing values and also reducing the risks of runtime errors that reduced compute power, which enabled the deployment of Edge compatibility for the future.
- **Scaling the Features:** Similar to the previous two points is Scaling the Features, which had ensured that the data was generally uniform with no extreme outliers, which had reduced the compute power in processing the data, and which had ensured easy Edge potential compatibility.
- **Applying Log Transformations:** Similar to the previous points, large data had been scaled appropriately, reducing computational expenses, which had enabled easy Edge potential compatibility.
- **SMOTENN:** By focusing on under-represented samples at local neighbourhoods, less noise is generated, less skewness had occurred, where this had resulted in less computational expenses, which had enabled easy Edge potential compatibility.
- **Pearson and Spearman Correlation Filtering:** By reducing highly correlated features and finding the ones that had underlying causes were identified, again this had reduced computational costs of processing the data, which had enabled easy Edge potential compatibility.
- **Stratified K-Fold CV and RFECV:** Retained the original distribution of the three combined Datasets, reducing uneven distributions, reducing the chances of runtime errors and/or computational expenses (Geeksforgeeks Editors 2025j). Furthermore, RFECV retained only the most important features, again reducing computational expenses, which had enabled easy Edge potential compatibility.
- **Isolation Forest:** Similar to the previous point, but for Standard Scalar scaled data only.
- **CNN Model:** As mentioned earlier within this Project Dissertation Batch Normalisation reduces basically data distribution of the activations and shift, which had resulted in training stabilisation, faster convergence, and less computational expenses, as well as improved gradient flow of Backpropagation, where the weights of the CNN Nodes are

adjusted according between the predicted and actual values, which had reduced computational expenses as well. Furthermore, Dropout Regularisation is used to disregard specific Nodes of the CNN Model, which had reduced overfitting, improved generalisation, and had boosted convergence rates (Wu et al. 2015). Additionally, Convolutional Layers were applied to assess feature importance and pass it down essentially into a single vector, which improved model efficiency, and this was similarly applied to a Flatten Layer. Regularisation of L1 and L2 was also applied to essentially reduce the complexity of the model by reducing the weights of the Nodes, improving generalisability, which had reduced computational costs. All in all, all of these techniques had enabled easy Edge potential compatibility. **LSTM Model:** Similar to the CNN model in terms of using Dropout Regularisation. Additionally, fewer layers were used compared with CNNs, and since the data was iteratively and sequentially updated, no use of very old data was required to be used again, which had overall reduced computational expenses. All in all, this had enabled easy Edge potential compatibility.

4 Implementation and Evaluation of Results

4.1 Implementation

The feasibility was ultimately determined by taking active steps to produce and eventually deploy the final model.

4.1.1 Importing the required Python Libraries

Specific grouping of files is used that are colloquially known as Modules that contains certain functions. Some contain certain standardised data values or for referenced language within produced code. The common libraries that were used for this were Pandas for Data Analysis, Matplotlib for Data Visualisation, sklearn that uses ML resources for predictive Data Analytics, and NumPy for numerical calculations, especially for random number generation and when using Mathematical Functions.

Most of these Imported Libraries are intuitive, so do not need any explaining (please refer to the Google Colab Python file), but some do. Within this particular context the Libraries that were used that needs explaining are the following:

- **!pip install-quiet kaggle pandas scikit-learn:** Was used for the combination of the three required Datasets, by installing the Kaggle Application Programming Interface (API) and installing the main Data Science Package. The quiet part restricts excessive output (pandas development team 2023a) (Team 2023b) (Pedregosa et al. 2011).
- **import os:** Working with the Operating System of Google Colab IDE when combining the three required Datasets (Foundation 2023) (Kaggle n.d.[b]).
- **from glob import glob:** Ensured matched file paths are matched when combining the three required Datasets (Foundation 2023).
- **!pip install imbalanced-learn (OPTIONAL):** Was optional for handling Class Imbalance, subsequently installing SMOTENN as an extra precautionary step (imbalanced-learn developers 2023).

- **from imblearn.combine import SMOTENN**: As explained above. Subsequently, an optional fallback logic can be used to prevent different versions and layout being used, as Python continuously updates these Libraries into new formats (imbalanced-learn developers 2023).
- **from collections import Counter**: Was used for rapid counting of discrete items, vital for ML Classification Label Distribution checks (Foundation 2023).
- **import random**: A Pseudo Random Number Generator for shuffling and reproducibility for the models that were created (Foundation 2023).
- **from collections import deque**: Was used due to being a queue in both directions for Buffers used in the RL and Hybrid DL-RL Models (Foundation 2023).
- **!pip install scikeras**: Used for installing Keras Models that acts a bridge to mimic scikit-learn estimators by wrapping Keras for GridSearchCV to be used for Hyperparameter Tuning to find the best DL Model, and for scoring CV (Pedregosa et al. 2011) (scikit-learn developers 2023).
- **from sklearn.model_selection import GridSearchCV**: As explained above (Pedregosa et al. 2011) (scikit-learn developers 2023).
- **from sklearn.linear_model import LogisticRegression**: Used for DQN Probabilities of new Q-Value States (Pedregosa et al. 2011) (scikit-learn developers 2023).
- **from sklearn.ensemble import RandomForestClassifier**: Used Random Permutations for RFECV rather than based on pureness for interpretability (Pedregosa et al. 2011) (scikit-learn developers 2023).
- **from sklearn.ensemble import IsolationForest**: Was used as an extra precaution to the StandardScaled Data RFECV to highlight the most important Features (not included within the final Model). Random partitioning is employed of Isolation Features, making anomalies easier to separate because they require fewer splits (Lemaître et al. 2017) (imbalanced-learn developers 2023).
- **from tensorflow.keras import layers, models, regularizers, callbacks**: This was used for the DL Models basically, explained in Section 3.3 (Abadi et al. 2016) (Chollet et al. 2015) (Team 2023c).
- **Keras Tuner (keras_tuner)**: The automation process for Hyperparameter Tuning being used (O'Malley et al. 2019)(Team 2023a).
- **from scikeras.wrappers import KerasClassifier, KerasRegressor**: Was used to wrap a Keras Model so it behaves like an sklearn estimator (Team 2023b).
- **import plotly.express as px**: Was used for plots and visual EDA (Inc. 2023).
- **import seaborn as sns**: Was used for Statistical EDA in addition to matplotlib (Waskom et al. 2017) (Inc. 2023) (Foundation 2023).

4.1.2 Loading the Acquired Datasets

Now once the Libraries have been imported, the foundations of any models is the Data itself, which was loaded as follows:

- **Uploading the Kaggle API Credentials: print(“Please upload your kaggle.json file:”) files.upload()**: The kaggle.json contains the Kaggle API token (username

- + key). This authenticates the user for programmatic access to all the available Kaggle Repository Datasets. This occurred via the `files.upload()`, where the json is manually uploaded, which is stored temporarily in the Colab working directory (Kaggle n.d.[a]) (Kaggle n.d.[c]) (Geeksforgeeks Editors 2025e).
- **Configuring the Kaggle Command Line Interface (CLI):** `os.makedirs(os.path.expanduser('~/.kaggle')!cpkaggle.json ~/.kaggle/!chmod600 ~/.kaggle/kaggle.json`: This ensured that the json file is properly placed in the expected path `~/.kaggle/` with correct permissions. The Kaggle CLI won't work if the file is missing or world-readable. `os.makedirs(..., exist_ok=True)` had ensured `~/.kaggle` existed, and `cpkaggle.json ~/.kaggle/` had copied the manually uploaded credentials, and finally `chmod 600` had set rigorous owner read/write permissions (Kaggle n.d.[c]) (Geeksforgeeks Editors 2025e).
 - **Downloading and Unzipping Datasets:** `dirs = 'ids_intrusion' : 'solarmainframe/ids - in os.makedirs(f"data/folder", exist_ok = True)!kaggleDatasetsdownload -d Dataset - pdata/folder --quietforarchiveinglob(f"data/folder/*.zip") :!unzip -oarchive - ddata/folder > /dev/null`: Was used for the automation of bulk downloading and unzipping Datasets from Kaggle into a structured folder hierarchy. `dirs` Was used to be a dictionary that maps the Dataset names to the ones found in Kaggle, `os.makedirs` was also used to ensure a readable subfolder for each Dataset, `!kaggle Datasets download -d Dataset` was also used to collect the Dataset zip, and finally `glob` finds all the stored zip files with `unzip -o` extracts them quietly behind the scenes (Kaggle n.d.[c])(Kaggle n.d.[b]) (Kaggle n.d.[a]) (Geeksforgeeks Editors 2025e).
 - **Defining NSL-KDD Dataset Columns:** The NSL-KDD Dataset did not come with column names, so it was required to name the manually (41 Features + 1 Label). It covers Basic Features such as duration, protocol, service, etc, Content Features such as failed logins, root access, etc, Traffic Features such as connection counts, and finally a Label (attack (Malign)/normal (Benign)) (please refer to the Google Colab Python file for the actual code itself) (Tavallae et al. 2009b) (Repository 1999).
 - **Loading NSL-KDD Train/Test:** Was used for loading both training and test splits, attach headers, and concatenate into a single DataFrame for preprocessing. `pd.read_csv(..., names=...)` Assigns the manual column names, and `pd.concat(..., ignore_index=True)` merges and resets indexing (please refer to the Google Colab Python file for the full actual code itself) (pandas development team 2023b).
 - **Loading CIC IDS-Intrusion 2018 Dataset :** There were multiple CSV files, so `glob` finds all of them. List comprehension + `pd.read_csv` loaded them efficiently, and finally `pd.concat` merged them into one Dataset for analysis. **NOTE: FOR THE LOADING OF THE CIC DDoS 2019 Dataset IS VERY SIMILAR TO THIS ONE.** (please refer to the Google Colab Python file for the full actual code itself) (Sharafaldin et al. 2019).

4.1.3 Data Pre-processing: Aligning Columns, Mapping Labels, Dataset Vertical Aggregation, Removing Redundant Columns, Preservation of Timestamps, One-Hot Encoding of each Dataset Categorical Columns, One-Hot Encoding all of the Labels, Sorting the Datasets, Saving the final combined Dataset

- **Aligning the columns:** Was used to had ensured that all three Datasets share the same Feature Schema. Missing features were also filled with `pd.NA`, where

both of these steps ensured that these heterogeneous Datasets combinable (please refer to the Google Colab Python file for the full actual code itself) (Rahman et al. 2020).

- **Automatic Labelling Mapping:** Was used to automatically convert diverse Text Labels such as normal, benign, malicious, ddos, etc into a Binary Encoding (0 = benign, 1 = malign).

The purposes were that these had enabled the handling of heterogeneous label naming, converted text to lowercase, stripped spaces, and detected benign indicators, and finally simplified downstream ML tasks (please refer to the Google Colab Python file for the full actual code itself) (Ring et al. 2019).

- **Combining the Datasets:** Was used to vertically merge the Datasets after alignment, because the Features are dynamic and independent of one another, so vertical instead of horizontal merging was used. This had in turn produced a larger, more diverse training corpus, improving generalisability of the models that were used (please refer to the Google Colab Python file for the full actual code itself) (Shiravi et al. 2012).
- **Dropping ID-like Columns:** Was used to remove identifiers such as IP addresses, flow IDs, ports, etc that may violate privacy regulations such as GDPR, etc as well as kept only behavioural/network features useful for detection (please refer to the Google Colab Python file for the full actual code itself) (Sommer et al. 2010).
- **Handling Timestamps:** Was used to normalise timestamps to a single column (Time), converting all formats into a standard datetime object, and finally was useful for sorting and temporal analysis (please refer to the Google Colab Python file for the full actual code itself) (Cordero et al. 2021).
- **One-Hot Encoding (Categorical Features + Labels):** Was used to convert Categorical Values into Binary Vectors such as protocol types, services, etc as well as had ensured the models can process non-numeric data, and for the Labels being One-hot Encoded into Label_0 (benign) and Label_1 (malign) (please refer to the Google Colab Python file for the full actual code itself) (Han et al. 2011).
- **Sorting and Saving:** Was used for sorting the chronological integrity provided that temporal models were used as well as ensured that the final Dataset was exported for reproducibility and later modelling (please refer to the Google Colab Python file for the full actual code itself) (He et al. 2009).

4.1.4 Data Pre-processing: EDA

- **Label Distribution:** Was used to ensure only the actual designated labels are being used from Pre-processing, aggregating for .sum() of the One-hot Encoded Binary Labels with summing providing the total count of benign and malign samples. Reindexing had also guaranteed consistent ordering (Label_0, Label_1), even if one class were missing in the current Dataset Split. The Labels were also renamed to improve interpretability-Label_0 for Benign and Label_1 for Malign (please refer to the Google Colab Python file for the full actual code itself) (Chawla et al. 2002).
- **Visualisation of Label Counts:** A Bar Plot was plotted with annotated Labels for a quick visual confirmation of Dataset balance or imbalance, and labelled exact count for improved clarity (please refer to the Google Colab Python file for the full actual code itself) (Chawla et al. 2002).

- **Histograms of Numeric Features:** Numeric Feature Selection was used to focus only on continuous/discrete numeric features (excluding labels). Then, Cleaning was used for replacing infinity values with NaN, and dropping missing values to avoid distortion. Subsequently, Conditional Plotting was then used for skipping constant columns (with only one unique value), since they add no statistical or ML value. The purpose of Histograms were to reveal the distribution shapes such as normal, skewed, tail-shape, kurtosis, etc, which helped inform choices such as Normalisation Standardisation, and/or Feature Engineering (please refer to the Google Colab Python file for the full actual code itself) (Tukey 1977) (Kirk 2016).
- **Defining Features (X) and Labels (y):** Feature-Label Separation had occurred: X = independent variables (Features), and y = dependent variable (target: 1 = malign, 0 = benign). This had ensured everything was in a numerical format coercing bad values (pd.NA, strings) into NaN. This was vital since scikit-learn models require numerical data (please refer to the Google Colab Python file for the full actual code itself) (Kuhn et al. 2013).
- **Hierarchical Splitting:** An 80%-10%-10% Train/Validation/Test Split was used to keep the majority for Training, while reserving 20% for Validation/Testing. The second split (10% Validation, 10% Test) had split the temp evenly. Subsequently, Stratification was used to preserve the Class Distribution within each subset, which is vital especially when these particular Datasets are imbalanced (Kohavi 1995) (Hastie et al. 2009). Finally a Random Seed of 42 was implemented, which had ensured. The number 42 is a drawn inspiration from the book title A Hitch Hiker's Guide to the Galaxy, where the number 42 seems to appear a lot in the Universe (please refer to the Google Colab Python file for the full actual code itself).
- **Conversion to NumPy Arrays:** Was used to convert Pandas DataFrames into NumPy arrays for compatibility with ML/DL libraries such as TensorFlow, scikit-learn, PyTorch, etc, and for Feature Scaling, Dimensionality Reduction, and Model Training (please refer to the Google Colab Python file for the full actual code itself) (Pedregosa et al. 2011).

4.1.5 Data Pre-processing: Imputation, Feature Scaling, and Log Transformation on Scaled Features

- **Mean Imputation:** Was used to handle missing values, particularly within tabular data (please refer to the Google Colab Python file for the full actual code itself) (Pedregosa et al. 2011) (Little et al. 2019).
- **Z-Score Normalisation:** Standardisation had transformed the Features to have mean 0 and variance 1, improving convergence in the implemented models (please refer to the Google Colab Python file for the full actual code itself) (Ioffe et al. 2015) (Kuhn et al. 2013).
- **Min-Max Scaling:** Had Scaled Features to a fixed range (typically [0,1]), which was useful for the models sensitive to Feature Magnitudes (please refer to the Google Colab Python file for the full actual code itself) (Han et al. 2011).
- **Verifying Split Shapes:** Verified consistency in split shapes that avoided mismatched errors before model training (please refer to the Google Colab Python file for the full actual code itself) (Goodfellow et al. 2016a).

- **Log Transformation:** Reduced skewness, stabilised variance, and made heavy-tailed distributions (large kurtosis) more Gaussian-like (please refer to the Google Colab Python file for the full actual code itself) (Box et al. 1964) (Tukey 1977).

4.1.6 Data Pre-processing: SMOTEENN

- **Subsampling:** Used 200k rows to prevent crashes. Additionally, stratification preserved class distribution in the subsample (please refer to the Google Colab Python file for the full actual code itself) (Altalhan et al. 2025).
- **SMOTE with reduced neighbours:** Generated synthetic minority samples by interpolating between k-nearest neighbours. Additionally, placing `k_neighbors=3` enables efficiency and lower likelihood of oversampling noisy regions (please refer to the Google Colab Python file for the full actual code itself) (Batista et al. 2004) (Algorithms 2025).
- **SMOTEENN:** Ideal for Class Imbalances.
- **Fitting SMOTEENN and resampling:** Applied SMOTEENN to the subsampled training data, producing a balanced and cleaned Dataset (please refer to the Google Colab Python file for the full actual code itself).
- **Printing before and after counts:** Had showed lass count pre- and post-resampling, which had verified balance (please refer to the Google Colab Python file for the full actual code itself).
- **Repeating procedure with MinMax-Scaling:** The whole workflow was similarly applied to the MinMax-Scaled Dataset (`X_train_minmax`) to compare the impact of scaling on resampling effectiveness (please refer to the Google Colab Python file for the full actual code itself).
- **Converting results to DataFrame/Series:** Converted the resampled arrays into structures that preserved Feature Names, which was vital for subsequent Feature Selection and/or interpretability (please refer to the Google Colab Python file for the full actual code itself).

4.1.7 Data Pre-processing: Pearson and Spearman Correlation Filtering

- **Computing Pearson Correlation:** Measured linear relationships between features that were sensitive to scale and outliers, where it was of use in detecting redundant linear dependencies (please refer to the Google Colab Python file for the full actual code itself) (Rodgers et al. 1988).
- **Extracting the upper triangle of the Correlation Matrix:** The matrices were symmetric by eliminating the lower triangle of the matrix, retaining only the unique feature pairs with the pairs only considered once (please refer to the Google Colab Python file for the full actual code itself).
- **Pearson:** Identified columns where correlation exceeded 0.95 with any other feature. 0.95 was selected due to commonly being identified as severe multicollinearity (please refer to the Google Colab Python file for the full actual code itself) (Dormann et al. 2013).
- **Spearman Correlation Filtering:** Measured the monotonic (ranked) relationships, which was less sensitive to outliers and non-linear transformations, where

non-linear relationships were captured, thus complemented Pearson Correlation Filtering (please refer to the Google Colab Python file for the full actual code itself) (Hauke et al. 2011).

- **Combining results from Pearson and Spearman:** Ensured the final combined union set dropped all the redundancies. The same workflow is repeated for MinMax-scaled features (X_minmax.df) (please refer to the Google Colab Python file for the full actual code itself).
- **Logging results:** Ensured transparency within Pre-processing, and kept note of the amount of dimensionality reduction that had occurred (please refer to the Google Colab Python file for the full actual code itself).

4.1.8 Data Pre-processing: RFECV

- **Ensuring Feature Alignment:** Once MinMaxScaling had been conducted, as well as Correlation Filtering, and SMOTEENN, the Feature Matrix (X) and Labels (y) were re-aligned. `.ravel()` Ensured y is 1D (required by the Python Library scikit-learn) (please refer to the Google Colab Python file for the full actual code itself) (Guyon, Weston et al. 2002) (Pedregosa et al. 2011).
- **Defining the Estimator:** A Random Forest Model (Decision Trees with a mean average voting system) for Feature Importance. The following chosen parameters were `n_estimators=100` (used 100 trees that was a sound trade-off and balance between accuracy and speed), `random_state=42` (as explained before, for reproducibility), and finally `n_jobs=-1` (for parallel processing) (please refer to the Google Colab Python file for the full actual code itself) (Guyon, Weston et al. 2002) (Pedregosa et al. 2011).
- **Cross-Validation Strategy:** Used Stratified 5-fold CV, which had ensured class balance in each fold, which was important for the imbalanced Datasets Geeksfor-geeks Editors (2025j), and `shuffle=True` randomised the folds (please refer to the Google Colab Python file for the full actual code itself) (Guyon, Weston et al. 2002) (Kuhn et al. 2013) (Pedregosa et al. 2011).
- **Running RFECV:** Repeatedly eliminated features, retrained the model, and evaluated the performance. The following parameters that had ensured this were `step=1` (one feature was removed iteratively), `scoring='balanced_accuracy'` (a fair metric for the imbalanced Data, especially for this particular Binary Classification, where the Labels are imbalanced). The output parameters were `rfecv.n_features_` (optimal number of Features) and `rfecv.support_` (a Boolean Mask of the selected Features) (please refer to the Google Colab Python file for the full actual code itself) (Guyon, Weston et al. 2002) (Kuhn et al. 2013) (Pedregosa et al. 2011).
- **Extracting the Best Features and forcing more Features if too few are selected:** Kept only the selected Features (not too few), and resulted in a smaller Feature Matrix optimised for classification (please refer to the Google Colab Python file for the full actual code itself) (Guyon, Weston et al. 2002) (Kuhn et al. 2013) (Pedregosa et al. 2011).
- **Visualising Feature Selection Performance:** Plotted CV performance against number of Selected Features. It was an aid in determining whether performance improved by dropping Redundant Features (please refer to the Google Colab Python file for the full actual code itself) (Pedregosa et al. 2011).

- **Repeating for Standard Scaler Features:** The exact same entire RFECV process was applied to the Standard Scaled Dataset (please refer to the Google Colab Python file for the full actual code itself) (Guyon, Weston et al. 2002) (Kuhn et al. 2013) (Pedregosa et al. 2011).

4.1.9 Data Pre-processing: PCA

- **Executing PCA:** A linear reduction technique that had found the orthogonal axes (Principal Components) that maximised variance. A 2D plot that had consisted of 2 components was plotted. This was applied both to MinMax-Scaled and Standard-Scaled Features (please refer to the Google Colab Python file for the full actual code itself) (Jolliffe et al. 2016).
- **Visualisation of the DataFrame:** The embedded PCA was converted to a DataFrame, where PCA 1 and PCA 2 were assigned as the two Principle Components respectively, and the Label as already explained previously. The Labels were assured to be integers for colouring visualisation (please refer to the Google Colab Python file for the full actual code itself).
- **Interactive Scatter Plot with Plotly:** Was used for an interactive 2D Scatter Plot. Each point represented network traffic event projected onto the PCA space, and Label colouring enabled visual inspection of separation (please refer to the Google Colab Python file for the full actual code itself) (Sievert 2020).
- **Repeating PCA for Standard-Scaled Data:** The same process, but for Standard-Scaled Features, providing a rigorousness check: providing that both scalings show similar separation patterns in PCA, it suggests scaling choice does not affect structure in variance space (please refer to the Google Colab Python file for the full actual code itself).

4.1.10 Data Pre-processing: Isolation Forest

- **Defining the specific Isolation Forest:** An anomaly detection algorithm that works by repeatedly splitting the data points, where anomalies are faster isolated. The specific parameters used were `n_estimators=100` (number of initial probability trees, where the greater the number of trees means greater stable anomaly detection), `contamination='auto'` (the automation of estimating the proportion of anomalies, instead of manually entering specific parameters), and finally `random_state=42`, which ensured reproducibility (please refer to the Google Colab Python file for the full actual code itself) (Chandola et al. 2009) (Liu et al. 2008).
- **Fitting on RFECV-Filtered Training Data:** Fitted the Isolation Forest model only on Training Data that was already reduced using RFECV Feature Selection. The output produced is assigned to two values: 1 = normal observation, and -1 = anomaly (outlier) (please refer to the Google Colab Python file for the full actual code itself) (Chandola et al. 2009).
- **Creating a Boolean Mask for the Anomalies:** Created a mask that identified all rows flagged as anomalies with all rows flagged as anomalies that gave the total anomaly count (please refer to the Google Colab Python file for the full actual code itself) (Chandola et al. 2009).

- **Filtering Out Anomalies:** Anomalous results were removed from both the Feature set (X_train_cleaned) and the Labels (y_train_cleaned). This was enabled by the use of outlier_mask (logical NOT) to keep only normal rows (please refer to the Google Colab Python file for the full actual code itself) (Chandola et al. 2009).

4.1.11 CNN Model

- **CNN Data Preparation:** The Feature Matrix was converted to float32 numerical format for TensorFlow compatibility. A Channel Dimension was also added for the CNN Model to work. channel=1 Because the Tabular NumPy Features are treated as a 1D sequence. The labels were also converted to int32 for Binary Classification (please refer to the Google Colab Python file for the full actual code itself) (Kiranyaz et al. 2021).
- **Train–Validation Split:** Ensured the CNN Model had Validation Data for Tuning and Early Feedback, with Stratification that had preserved Class Balance (please refer to the Google Colab Python file for the full actual code itself) (Kiranyaz et al. 2021). **CNN Model Builder with Hyperparameters:** Used 32 and 64 Convolutional Filters with 2 or 3 Kernel Size (compatible with the Tabular Features), L2 Regularisation with a Learning Rate. The Architectural Design was Convolutional 1D Filter, Batch Normalisation, Dropout, another Convolutional 1D Filter, Batch Normalisation, MaxPooling, and Dropout with Flatten and Dense Layers with L1/L2 Regularisation, and finally a final Sigmoid Output for Binary Classification. The reasons were that Batch Normalisation had stabilised the Training with faster convergence, Dropout to prevent highlight similar Nodes co-adapting, L1/L2 Reg. for sparse Data and Smaller Weights for finer details, all collectively enhancing generalisation (please refer to the Google Colab Python file for the full actual code itself) (Kiranyaz et al. 2021)(O’Malley et al. 2019)(Srivastava et al. 2014) (Wu et al. 2015).
- **Hyperparameter Tuning with Keras Tuner:** Performed Random Search across possible Hyperparameters. 5 Trials were utilised for speed and efficiency, with each training a new CNN variant. The best model was selected based on the performance metric of Validation Accuracy (please refer to the Google Colab Python file for the full actual code itself) (O’Malley et al. 2019).
- **Evaluation on Validation Data:** Predicted probabilities for the Sigmoid Output with a threshold equal to 0.5 resulting in a conversion to Binary Predictions. Evaluations of Precision, Recall, F1-Score, Accuracy, and ROC-AUC score (apart from Accuracy, all the rest were to do with True/False Positives) (please refer to the Google Colab Python file for the full actual code itself) (Fawcett 2006).
- **Final Training of Best Model:** Trained the selected CNN Model for 20 Epochs, and Stored the History for plotting Learning Curves (please refer to the Google Colab Python file for the full actual code itself).
- **Performance Visualisation:** An ROC Curve was plotted to portray the balance between True/False Positives rate, Accuracy Curve for progress monitoring, and finally a Loss Curve to monitor convergence and overfitting (please refer to the Google Colab Python file for the full actual code itself) (Fawcett 2006).

4.1.12 LSTM Model

- **Data Preparation:** The Features were reshaped to the required 3D format that was required for the LSTM model to work (samples, timesteps, features_per_step). The Tabular Data was regarded as a timestamp with one channel with the Labels `y_train`, `y_val`, `y_test` remained as 1D Arrays (please refer to the Google Colab Python file for the full actual code itself) (Hochreiter et al. 1997a) (Chollet 2017).
- **Building the LSTM Model with Hyperparameter Tuning:** Consisted of 2 LSTM hidden layers with changeable hidden units within the inclusive range of 32-128, a Dense Layer for added Feature Transformation before Output, Dropout Layers to improve generalisability, and finally Tuning the Learning Rate for convergence rate control (please refer to the Google Colab Python file for the full actual code itself) (Greff et al. 2017) (O'Malley et al. 2019) (Wu et al. 2015).
- **Compiling the Model:** Used an Adam Optimiser for Adaptive Learning Rate for Stable Training, Binary Cross Entropy Loss for Binary Classification, and finally Accuracy as a Metric as the main Evaluation for Training (please refer to the Google Colab Python file for the full actual code itself) (Kingma et al. 2015).
- **Hyperparameter Tuning with Random Search:** Used Random Search to explore various Hyperparameter combinations with the aim of obtaining the best Validation Accuracy (please refer to the Google Colab Python file for the full actual code itself) (Bergstra et al. 2012).
- **Evaluation:** The predictions were probabilistic with a threshold of greater than 0.5 resulting into the conversion into integer format. Subsequently, these were evaluated with Precision, Recall, F1-Score, and a ROC-AUC Score for realistic Accuracy (please refer to the Google Colab Python file for the full actual code itself) (Powers 2011).
- **Training and Visualisation:** Trained the best LSTM Model with the best Hyperparameters with the History recording and storing the Accuracy and Loss Curves for Training against Validation, and plotted an ROC Curve for performance balancing (please refer to the Google Colab Python file for the full actual code itself) (Fawcett 2006).

4.1.13 DQN Model

- **Data Preparation:** Was used for the conversion of the Feature Matrix into a NumPy array suitable for feeding into the Neural Network. The Labels were explicitly converted into integers to enable classification. The Validation and Test sets were aligned to retain the same Feature Columns after Correlation Filtering and RFECV (please refer to the Google Colab Python file for the full actual code itself) (Kuhn et al. 2013).
- **Ensuring Both Classes Exist:** RL assumes both of these classes exist, so a dummy sample was inserted if the Training Data had only one class. This prevented the model from collapsing to a single-class policy (please refer to the Google Colab Python file for the full actual code itself) (He et al. 2009).
- **DQN Hyperparameters and Replay Buffer:** Consisted of a Discount Factor ($\gamma = 0.99$) that had controlled the importance of future Rewards, an Epsilon-Greedy Policy that balanced exploration (random actions) against exploitation

(best Q-values), and a Replay Buffer (memory) that stored past Experiences to break Correlation and stabilise Training (please refer to the Google Colab Python file for the full actual code itself) (Lin 1992) (Mnih et al. 2015).

- **Building the DQN Model:** Used a Feed Forward Neural Network to approximately calculate the Q-function. Hidden layers comprised of 32 and 64 units with RELU Activations were used respectively with a Linear Activations Output Layer, producing Q-values for each Action (please refer to the Google Colab Python file for the full actual code itself) (Goodfellow et al. 2016b).
- **Epsilon-Greedy Action Selection:** Used Random Action with Probability ϵ (Exploration), and Greedy Action with Probability $(1 - \epsilon)$ (Exploitation) with a gradual shift towards Exploitation as the Training progresses (please refer to the Google Colab Python file for the full actual code itself) (Sutton et al. 2018).
- **Training Loop:** Experiences (state, action, reward, next_state, done) were sampled from Memory. The Q-values were then updated using the Bellman Equation: $Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$, where the Targets (Labels) were constructed and the model was Trained using MSE (please refer to the Google Colab Python file for the full actual code itself) (Watkins et al. 1992).
- **Calibration with Logistic Regression:** Was used since Q-values were not inherently calibrated values, so a Logistic Regression Model resolved this by mapping them to well-calibrated outputs (please refer to the Google Colab Python file for the full actual code itself) (Niculescu-Mizil et al. 2005).
- **Evaluation:** Used the metrics of Accuracy, Precision, Recall, F1, ROC-AUC with a Confusion Matrix to visualise misclassifications, and finally ROC curves to compare Validation against Test Performance (please refer to the Google Colab Python file for the full actual code itself) (Powers 2011).
- **Visualisation:** Used to monitor Training loss over Episodes of Learning Stability. ROC and Confusion Matrix gave insights into Classifier Quality (please refer to the Google Colab Python file for the full actual code itself) (Fawcett 2006).

4.1.14 CNN-DQN Model

- **CNN Model Building with Keras Tuner:** Used to define a 1D CNN architecture using the Functional API that applied Convolution, Batch Normalisation, Dropout, Max Pooling, and Dense Layers. The hyperparameters (Filters, Kernel Size, L2 Penalties, Learning Rate) are Tuned with Keras Tuner (please refer to the Google Colab Python file for the full actual code itself) (O'Malley et al. 2019)(LeCun et al. 2015) (Ioffe et al. 2015) (Srivastava et al. 2014) (Wu et al. 2015) (Ng 2004).
- **Hyperparameter Search with RandomSearch:** Used to run various random searches for different CNN configurations to optimise Validation Accuracy (please refer to the Google Colab Python file for the full actual code itself) (O'Malley et al. 2019).
- **Extracting Embeddings from the CNN Encoder:** Used the penultimate layer of the CNN Model as a Feature Encoder, which converted the Features directly into Embeddings, which served as the input for the DQN Model (please refer to the Google Colab Python file for the full actual code itself) (LeCun et al. 2015).

- **Class Balance Check:** Ensured that both Classes existed in the Training Data. If missing, a Dummy Sample was added, which avoided errors during Training (please refer to the Google Colab Python file for the full actual code itself) (Powers 2011).
- **DQN Hyperparameters and Replay Buffer:** Defined the DQN Configuration with the State Size representing the CNN Embedding Dimension, the Action Size being the Binary Classification as discussed before, and the Replay Buffer which stored the Transitions for Experience Replay (please refer to the Google Colab Python file for the full actual code itself) (Mnih et al. 2015).
- **DQN Model (MLP after CNN embeddings):** Used an MLP that mapped the Embeddings to the Q-values (please refer to the Google Colab Python file for the full actual code itself) (Mnih et al. 2015).
- **Training Loop with Epsilon-Greedy Policy:** Implemented this Policy of Random Actions with probability ϵ conducted Exploration, otherwise Exploited learned Q-values (please refer to the Google Colab Python file for the full actual code itself) (Mnih et al. 2015).
- **Replay Buffer Updates and Target Calculation:** Updated the Q-Value Targets that used the Bellman equation (Mnih et al. 2015).
- **Calibration with Logistic Regression:** Used a Logistic Regression Model that calibrated Q-value into probabilities (please refer to the Google Colab Python file for the full actual code itself) (Platt 1999).
- **Evaluation Metrics:** Gave the Accuracy, Precision, Recall, F1-Score, and ROC-AUC on the Validation and Test Sets (please refer to the Google Colab Python file for the full actual code itself) (Fawcett 2006)(Powers 2011).
- **Visualisation:** Plotted the Training Loss for Stabilisation monitoring, ROC Curves for Validation against the Test Sets comparison, and a Confusion Matrix Heatmap used as a Classification Report (please refer to the Google Colab Python file for the full actual code itself) (Fawcett 2006) (Powers 2011).

4.1.15 LSTM-DQN Model

- **Data Preparation for the LSTM-DQN Model:** RFECV Features were retained with MinMax Scaling that ensured all the Features were Normalised to a consistent range. The Labels were also converted to integers (please refer to the Google Colab Python file for the full actual code itself) (Hochreiter et al. 1997b)(Brownlee 2017).
- **LSTM Model Construction with Hyperparameter Tuning:** Used a Keras Functional API (ensured greater flexibility and modularity) for the LSTM Model construction with its Model Architecture consisting of two layers for capturing Long and Short-Term Temporal Dependencies, Dropout Layers for Regularisation of Overfitting, Dense Layers for learning higher-level representations, Sigmoid Output for Binary Classification, and with the typical choices of an Adam Optimiser and Binary Cross-Entropy Loss (please refer to the Google Colab Python file for the full actual code itself) (Bergstra et al. 2012)(Keras Team 2023).
- **Training and Validation:** Trained the Tuned LSTM Model with the output being Metric Reporting of the Accuracy, Precision, Recall, F1-Score, ROC-AUC

(typical for Binary Classification) with the ROC Curve visualising the trade-off between Sensitivity and Specificity, and Loss/Accuracy Curves to monitor Stabilisation and to see if overfitting or underfitting occurs (please refer to the Google Colab Python file for the full actual code itself) (Saito et al. 2015).

- **Encoder for Embeddings:** Used to repurpose the Trained LSTM Model to extract Features, capturing Latent Embeddings from the Dense Penultimate Layer before the Sigmoid Output Layer, which represented Input Data Compressed Temporal Patterns, which were used as inputs for the DQN Agent (please refer to the Google Colab Python file for the full actual code itself) (Bengio et al. 2013).
- **DQN Model:** Consisted of the State that contained the Embeddings, Reward System (+1 if predicted correctly, -1 if not), as well as Replay Experience and Epsilon-Greedy Exploration Policy, which were explained before (please refer to the Google Colab Python file for the full actual code itself) (Mnih et al. 2015).
- **Evaluation:** After training the DQN, predictions were calibrated with a Logistic Regression Model to improve probability estimates, with the recording of the Metrics of Accuracy, Precision, Recall, F1, ROC-AUC Scores, with ROC Curves and a Confusion Matrix that provided greater insight into Classification Performance and trade-offs between generalisation and regularisation (please refer to the Google Colab Python file for the full actual code itself) (Niculescu-Mizil et al. 2005).
- **Visualisation:** Used to plot the Train Loss over the Episodes for monitoring the DQN Agent's Convergence, ROC Curves and a Confusion Matrix as explained above in the previous point (please refer to the Google Colab Python file for the full actual code itself) (Kirk 2016).

4.1.16 xAI and LLM Integration

- **Dependency Installation and Imports:** Required for SHAP and LIME to be operational, for the request that were used to interact with the Hugging Face Inference API, and with the MinMaxScaler that ensured Feature Scaling before applying interpretability methods (please refer to the Google Colab Python file for the full actual code itself) (Lundberg et al. 2017)(Ribeiro et al. 2016a).
- **Data Preparation for XAI:** The LSTM Embeddings were Flattened back into Tabular Format so SHAP and LIME, which were feature-based, had operated, which ensured Feature Alignment across the Training, Validation, and Test Sets (please refer to the Google Colab Python file for the full actual code itself) (Molnar 2025).
- **SHAP Explanations:** Used a Background Sample as SHAP required a Background Dataset (Reference Distribution) to have Computed Shapley values with the output image being a KernelExplainer that was a model-agnostic method that had approximated Shapley values by Perturbation changes, and produced Feature Attrition Values that explained each Feature contributions to the model's decision (please refer to the Google Colab Python file for the full actual code itself) (Shapley 1953).
- **LIME Explanations:** Generated a locally interpretable model by changing the input and observing the predictions generated as a result, ranked Feature Importance based on these predictions, and complemented SHAP by offering individual explanations (please refer to the Google Colab Python file for the full actual code itself) (Guidotti et al. 2019).

- **Hugging Face ZySec-7B Integration:** A Cybersecurity specific LLM hosted on Hugging Face model repository, where the user provided their Hugging Face API token for authentication (please refer to the Google Colab Python file for the full actual code itself) (Wolf et al. 2020).
- **Prompt Engineering with XAI Results:** A Pipeline that integrated SHAP and LIME into a Natural Language Prompt (NLP), which was then give to ZySec-7B LLM that enabled human-understandable explanations with contextualised Cybersecurity insights (please refer to the Google Colab Python file for the full actual code itself) (Bommasani et al. 2021).
- **API Request and Response Handling:** Generated NLP response if proven successful, otherwise an error was raised if the API had failed (please refer to the Google Colab Python file for the full actual code itself) (Song, Xiong et al. 2023).

4.2 Evaluation of the Results

Episode / Metric	CNN-DQN Hybrid	LSTM-DQN Hybrid
Episode 1/3	Episode finished, $\epsilon = 0.995$	Episode finished, $\epsilon = 0.995$
Episode 2/3	Episode finished, $\epsilon = 0.990$	Episode finished, $\epsilon = 0.990$
Episode 3/3	Episode finished, $\epsilon = 0.985$	Episode finished, $\epsilon = 0.985$
Accuracy	0.9988	0.9988
Recall	1.0	1.0
Precision	0.9988	0.9988
F1 Score	0.9994	0.9994
ROC-AUC	0.5077	0.5000

Table 1: Final Hybrid Model Results: Comparison of CNN-DQN and LSTM-DQN

NOTE: The individual models had all the same Accuracy bar the CNN model. The CNN Model had 100% in every Metric, which is a cause for concern because this indicates massive overfitting and no generalisability of the model itself. The possible causes were due to the fact that all three Datasets were combined that had different dimensions, so cause misalignment, as well as Labels being so varied that standardising them to one uniform format also resulted in misalignment and reduced diversity that skewed and misrepresented the results. Additionally, SMOTEENN had created too much noise, which eliminated the Labels entirely as this was perceived to be noise, also with RFECV similarly initially only including one feature was due to misalignment of the three combined Datasets. Furthermore, since everything is 100% for Metric Performance of the CNN Model, this means all results are TPR so no ROC-AUC Score, again due to the reasons explained above. The LSTM Model was more realistic since it exactly matches the results of the metric of the LSTM-DQN Hybrid model but still suffers from the same concerns as explained above. The DQN Model also has every metric the same except the ROC-AUC Score, which is slightly less at approximately 0.48, where this underperformance of TPR and higher than usual FPR was due to the fact that since SMOTEENN wiped out the actual Labels, Dummy versions were used to compensate for this loss that were not representative. Also, it still suffered from all the other problems explained above. Now, regarding the Hybrid Models, except for the ROC-AUC result, everything else is the same. The reasons for this are exactly the same as

explained above. As well as, with the Episodes from the DQN RL Agent from the model and Hybrids Models themselves, were experiencing high randomness, so resulted in greater Exploration rather than Exploitation; this being due to the reasons as explained above, and also due to greater computational expenses and overheads, resulted in a minimal of 3 Episodes being utilised. Finally, with regard to xAI and LLM Integration, more research needs to be conducted as this is resource intensive and very time consuming, and is just a preliminary suggestion.

5 Discussion

As the Evaluation of the Results have been explained in the previous section, superficially these results look really good, but are very deceptive due to the Data not being aligned properly—remember that your models are as only as good as your Data that are being used. So, when considering if it matches the final model choice mentioned in the Systematic Literature Review, not in its entirety, which naturally leads on to the Future Improvements Suggestions Section.

6 Future Work to Incorporate Suggestions % Conclusions

6.1 Future Work to Incorporate Suggestions

The most glaringly obvious improvements are to run the Datasets separately, by modifying how the Datasets are loaded in a generic way such as reading a CSV file, and retain the same variable names for consistency within the code produced. Also, the NSL-KDD Dataset obtained from any publicly available Data Repository needs to be in an immediately readable and easy to understand format. Due to the computational expenses and time consumption, the number of Epochs (runnings) was limited to 20 for the CNN, LSTM, and Hybrid Models, respectively, as well as the number of Episodes for the RL DQN Agent. By increasing the computational power such as increasing the number of available GPU's, this will enable more runnings, which will improve the Performance Metrics, and improve the Exploration-Exploitation trade-off by increasing the number of Episodes for the RL DQN Agent, as the randomness will eventually decrease to a suitable level. Additionally, trying novel models can also be of use, providing potentially different Performance Metrics, for example as mentioned earlier within this Project Dissertation: GANs. These work by forming realistic data by training two separate Neural Networks within a competitive environment (Avijeet Biswal 2025). The components consists of a Network Generator, where counterfeit data is generated from Stochastic Noise, a Discriminator Network, which evaluates the realness of the data provided, separating real from fake, and the training steps themselves, which simultaneously the Generator and Discriminator are trained, with the Generator attempting to trick the Discriminator by producing good quality fake data as if it were real, and the Discriminator getting better at easily distinguishing fake data (Avijeet Biswal 2025). Another model being Transformer Networks, which forms the basis of many NLPs, processing data input by using a self-attention mechanism, enabling operations to run in parallel and better management if dependencies that are long-term (Avijeet Biswal 2025). They consist of a self-attention process which calculates the importance of the input/s compared to other components, allowing the model to assign various weights that represent the importance of different words within a given sentence, Encoding via Positions, where word positional

information is captured as the previous process does not measure sequence order, and the structure of Encoding-Decoding, where the inputs are Encoded, outputs Decoded, generating the results with multiple layers of Self-Learning and simple Feed-Forward Neural Networks (Avijeet Biswal 2025). Another model also similar to the previous one is an Autoencoder that uses the same Encoder-Decoder process as the previous model that is useful to represent all the data in a smaller format (Avijeet Biswal 2025). Similarly, another model is a Variational Autoencoders (VAEs), which is like an Autoencoder but generates completely new data points that are synthetic and are similar to the ones found in training, which are used for data imbalances, generative work, and outlier detection (Avijeet Biswal 2025). This model consists of the components of an Encoder that works like the few previous models, a Latent Space for Sampling that brings variation in the data Bergman Dave (2025b), a Decoder that also works like the few previous models, and training to minimize the Decoding Reconstruction Loss and some form of Regularisation that causes the Latent Space Bergman Dave (2025b) to follow a Normal Distribution format (Avijeet Biswal 2025). Finally, the last suggested model and any future improvements is a Graph Neural Network (GNN) that forms a visual representation of a Neural Network, which is used a lot within the field of Network Science (Avijeet Biswal 2025). The model consists of Nodes representing the object of particular interest, Edges showing the relationships, passing information via the Edges to update their representations potentially, and a function that collects and combines the representations of the Nodes in order to produce the graph, which is suitable for classification tasks such as this, and for regression tasks such as the one used for the DQN Model, which was a Logistic Regression (Avijeet Biswal 2025).

6.2 Conclusions

To summarise this whole Project Dissertation with a final complete fix would be impossible since this Field is constantly evolving. Regarding the Aim and Objectives, this has not met them in its entirety, as due to the problems explained in the Evaluation and Future Work Sections. Finally, regarding the Scope and Considerations, while most of them have been met, more work needs to be done on the xAI and Vibe Coding side as explained before earlier on. To summarise the key findings, all in all, the Datasets need to be run individually, with the code slightly modified in regard to the Loading the Datasets part, more model testing, and more research conducted on the LLM Integration choice and Vibe Coding, setting the direction to eventually achieving a solid foundation for the integration of AI within IDS based frameworks for enhanced Network Security.

References

- Abadi, M. et al. (2016). ‘TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems’. Software available from tensorflow.org.
- Abdulhammed, R., Faezipour, M., Abuzneid, A. and AbuMallouh, A. (2019). ‘Feature selection and ensemble learning for improved anomaly detection in network traffic’. *Neural Computing and Applications* 31 (11), pp. 9105–9114. Available at: <https://doi.org/10.1007/s00521-019-04432-3>. URL: <https://doi.org/10.1007/s00521-019-04432-3>.
- Algorithms (2025). ‘SMOTE vs. SMOTEENN: A Study on the Performance of Resampling Algorithms for Addressing Class Imbalance in Regression Models’. *Algorithms (MDPI)* 18 (X), pp. YYY. Available at: <https://doi.org/https://doi.org/10.3390/algorithmsXXXXXX>. URL: <https://doi.org/10.3390/algorithmsXXXXXX>.
- Altalhan, M., Algarni, A. and Turki-Hadj Alouane, M. (2025). ‘Imbalanced Data Problem in Machine Learning: A Review’. *IEEE Access* 13, pp. 13686–13699. Available at: <https://doi.org/https://doi.org/10.1109/ACCESS.2025.3531662>. URL: <https://doi.org/10.1109/ACCESS.2025.3531662>.
- Amidi, Afshine and Amidi, Shervine (2025). ‘Recurrent Neural Networks cheatsheet’. Accessed: 20 September 2025. URL: <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>.
- Avijeet Biswal (2025). ‘Top 10 Deep Learning Algorithms You Should Know in 2025’. Accessed: 18 September 2025. URL: <https://www.simplilearn.com/tutorials/deep-learning-tutorial/deep-learning-algorithm>.
- Batista, G. E. A. P., Prati, R. C. and Monard, M. C. (2004). ‘A study of the behavior of several methods for balancing machine learning training data’. *SIGKDD Explorations Newsletter* 6 (1), pp. 20–29. Available at: <https://doi.org/https://doi.org/10.1145/1007730.1007735>. URL: <https://doi.org/10.1145/1007730.1007735>.
- Bengio, Y., Courville, A. and Vincent, P. (2013). ‘Representation learning: A review and new perspectives’. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (8). (Accessed: 19 September 2025), pp. 1798–1828. Available at: <https://doi.org/10.1109/TPAMI.2013.50>. URL: <https://doi.org/10.1109/TPAMI.2013.50>.
- Bergman Dave (2025a). ‘What is knowledge distillation?’ Accessed: 20 September 2025. URL: <https://www.ibm.com/think/topics/knowledge-distillation>.
- (2025b). ‘What is latent space?’ Accessed: 20 September 2025. URL: <https://www.ibm.com/think/topics/latent-space>.
- Bergstra, J. and Bengio, Y. (2012). ‘Random Search for Hyper-Parameter Optimization’. *Journal of Machine Learning Research* 13, pp. 281–305.
- Bommasani, R., Hudson, D. A., Adeli, E., ... and Liang, P. (2021). ‘On the Opportunities and Risks of Foundation Models’. arXiv preprint arXiv:2108.07258. Stanford CRFM report on foundation models; useful when discussing LLMs as infrastructure. (Accessed: 19 September 2025). URL: <https://arxiv.org/abs/2108.07258>.
- Box, G. E. P. and Cox, D. R. (1964). ‘An Analysis of Transformations’. *Journal of the Royal Statistical Society: Series B (Methodological)* 26 (2), pp. 211–252. Available at: <https://doi.org/https://doi.org/10.1111/j.2517-6161.1964.tb00553.x>. URL: <https://doi.org/10.1111/j.2517-6161.1964.tb00553.x>.
- Brownlee, J. (2017). ‘Long Short-Term Memory Networks With Python’. (Accessed: 19 September 2025). Machine Learning Mastery. URL: <https://machinelearningmastery.com/lstms-with-python/>.

- Chandola, V., Banerjee, A. and Kumar, V. (2009). ‘Anomaly detection: A survey’. *ACM Computing Surveys* 41 (3), pp. 1–58. Available at: <https://doi.org/10.1145/1541880.1541882>. URL: <https://doi.org/10.1145/1541880.1541882>.
- Chandrashekar, G. and Sahin, F. (2014). ‘A survey on feature selection methods’. *Computers and Electrical Engineering* 40 (1), pp. 16–28. Available at: <https://doi.org/10.1016/j.compeleceng.2013.11.024>.
- Charmet, F., Tanuwidjaja, H. C., Ayoubi, S., Gimenez, P.-F., Han, Y., Jmila, H., Blanc, G., Takahashi, T. and Zhang, Z. (2022). ‘Explainable artificial intelligence for cybersecurity: A literature survey’. *Annals of Telecommunications* 77 (10), pp. 789–812. Available at: <https://doi.org/10.1007/s12243-022-00926-7>.
- Chawla, N. V., Bowyer, K. W., Hall, L. O. and Kegelmeyer, W. P. (2002). ‘SMOTE: Synthetic Minority Over-sampling Technique’. *Journal of Artificial Intelligence Research* 16, pp. 321–357. Available at: <https://doi.org/10.1613/jair.953>.
- Chen, J., Zhou, H., Mei, Y., Adam, G., Bastian, N. D. and Lan, T. (2023). ‘Real-time network intrusion detection via decision transformers’. *arXiv preprint arXiv:2312.07696* 2, pp. 1–9. Available at: <https://doi.org/10.48550/arXiv.2312.07696>.
- Chollet, F. (2017). ‘Deep Learning with Python’. Shelter Island, NY: Manning Publications.
- Chollet, F. et al. (2015). ‘Keras’. GitHub repository: <https://github.com/keras-team/keras>.
- CLOUDFLARE Editors (2025). ‘What is quantization in machine learning?’ Accessed: 20 September 2025. URL: <https://www.cloudflare.com/en-gb/learning/ai/what-is-quantization/>.
- Cordero, C., Rivas, A., Herrero, Á. and Corchado, E. (2021). ‘A review on time-aware machine learning methods for cybersecurity’. *Future Generation Computer Systems* 115, pp. 768–783. Available at: <https://doi.org/https://doi.org/10.1016/j.future.2020.09.005>. URL: <https://doi.org/10.1016/j.future.2020.09.005>.
- Doost, P. A., Moghadam, S. S., Khezri, E., Basem, A. and Trik, M. (2025). ‘A new intrusion detection method using ensemble classification and feature selection’. *Scientific Reports* 15, pp. 13642. Available at: <https://doi.org/10.1038/s41598-025-98604-w>.
- Dormann, C. F. et al. (2013). ‘Collinearity: a review of methods to deal with it and a simulation study evaluating their performance’. *Ecography* 36 (1), pp. 27–46.
- Editors of Imperva (2025). ‘Site Scanning/Probing’. Accessed: 1 July 2025. URL: <https://www.imperva.com/learn/application-security/site-scanning-probing/#:~:text=Site%20scanning%20and%20probing%20are,specific%20weaknesses%20or%20gather%20intelligence..>
- ‘Effective Analysis on Remote to User (R2L) Attacks Using Random Forest Algorithm.’ (2014). Accessed: 1 July 2025. URL: https://www.academia.edu/7314391/Effective_Analysis_on_Remote_to_User_R2L_Attacks_Using_Random_Forest_Algorithm#:~:text=random%20forest%20algorithm.-,The%20R2L%20attack%20occurs%20when%20the%20attacker%20tries%20to%20send,secured%20information%20from%20the%20machine..
- Eke, H. and Petrovski, A. (2023). ‘Advanced Persistent Threats (AAM)’. In: *Proceedings of the 2023 International Conference on Electronics, Computers and Artificial Intelligence (EKE)*. [Accessed 25 June 2025]. Robert Gordon University. URL: [Available%20at:%20https://rgu-repository.worktribe.com/preview/1931374/EKE%202023%20Advanced%20persistent%20threats%20%28AAM%29.pdf](https://rgu-repository.worktribe.com/preview/1931374/EKE%202023%20Advanced%20persistent%20threats%20%28AAM%29.pdf).
- Ekky Kharismadhany (2018). ‘CICIDS2018 Cleaned’. Accessed: 31 August 2025. URL: <https://www.kaggle.com/datasets/ekkykharismadhany/csecicids2018-cleaned>.

- Fawcett, T. (2006). 'An introduction to ROC analysis'. *Pattern Recognition Letters* 27 (8). (Accessed: 19 September 2025), pp. 861–874. Available at: <https://doi.org/10.1016/j.patrec.2005.10.010>. URL: <https://doi.org/10.1016/j.patrec.2005.10.010>.
- Fedus, W., Ramachandran, P., Agarwal, R., Bengio, Y., Larochelle, H., Rowland, M. and Dabney, W. (2020). 'Revisiting Fundamentals of Experience Replay'. Accessed: 20 September 2025. URL: <https://arxiv.org/abs/2007.06700>.
- Ferrag, M., Maglaras, L., Derhab, A. and Janicke, H. (2022). 'Intrusion Detection and Prevention Systems for Cybersecurity: A Review'. *IEEE Access* 10, pp. 18185–18213. Available at: <https://doi.org/10.1109/ACCESS.2022.3146652>.
- FieryBanana101 (2019). 'CICDDos2019'. Accessed: 1 July 2025. URL: <https://www.kaggle.com/datasets/fierybanana101/cicddos2019>.
- Foundation, P. S. (2023). 'Python Documentation'. Accessed: 28 August 2025.
- GeeksforGeeks Editors (2025). 'Pearson vs Spearman Correlation Coefficient'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/maths/pearson-vs-spearman-correlation-coefficient/>.
- Geeksforgeeks Editors (2023). 'Epsilon-Greedy Algorithm in Reinforcement Learning'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/machine-learning/epsilon-greedy-algorithm-in-reinforcement-learning/>.
- (2025a). 'Backpropagation in Neural Network'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/>.
- (2025b). 'CNN — Introduction to Pooling Layer'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/deep-learning/cnn-introduction-to-pooling-layer/>.
- (2025c). 'Deep Q-Learning in Reinforcement Learning'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/deep-learning/deep-q-learning/>.
- (2025d). 'Dropout in Neural Networks'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/machine-learning/dropout-in-neural-networks/>.
- (2025e). 'How to Import Kaggle Datasets Directly into Google Colab'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/python/how-to-import-kaggle-datasets-directly-into-google-colab/>.
- (2025f). 'Internal Covariant Shift Problem in Deep Learning'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/deep-learning/internal-covariant-shift-problem-in-deep-learning/>.
- (2025g). 'Regularization in Machine Learning'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/machine-learning/regularization-in-machine-learning/>.
- (2025h). 'ReLU Activation Function in Deep Learning'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/>.
- (2025i). 'Sigmoid Function'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/machine-learning/derivative-of-the-sigmoid-function/>.
- (2025j). 'Stratified K Fold Cross Validation'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/machine-learning/stratified-k-fold-cross-validation/>.
- (2025k). 'Understanding of LSTM Networks'. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/machine-learning/understanding-of-lstm-networks/>.

- Geeksforgeeks Editors (2025l). ‘Vanishing and Exploding Gradients Problems in Deep Learning’. Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/deep-learning/vanishing-and-exploding-gradients-problems-in-deep-learning/>.
- (2025m). ‘What are Convolution Layers?’ Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/machine-learning/what-are-convolution-layers/>.
- (2025n). ‘What is Batch Normalization in CNN?’ Accessed: 20 September 2025. URL: <https://www.geeksforgeeks.org/computer-vision/what-is-batch-normalization-in-cnn/>.
- Ghubaish, A., Yang, Z. and Jain, R. (2024). ‘HDRLIDS: A Hybrid Deep Reinforcement Learning Intrusion Detection System for Enhancing the Security of Medical Applications in 5G Networks’. In: *Proceedings of the 2024 International Conference on Smart Applications, Communications and Networking (SmartNets 2024)*. Harrisonburg, VA, USA: IEEE. Available at: <https://doi.org/10.1109/SmartNets61466.2024.10577692>.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016a). ‘Deep Learning’. MIT Press. URL: <https://www.deeplearningbook.org/>.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016b). ‘Deep Learning’. MIT Press.
- Greff, K., Srivastava, R. K., Koutník, J., Steunebrink, B. R. and Schmidhuber, J. (2017). ‘LSTM: A Search Space Odyssey’. *IEEE Transactions on Neural Networks and Learning Systems* 28 (10), pp. 2222–2232. Available at: <https://doi.org/https://doi.org/10.1109/TNNLS.2016.2582924>.
- Gueriani, A., Kheddar, H. and Mazari, A. C. (2024). ‘Deep reinforcement learning for intrusion detection in IoT: A survey’. *arXiv preprint arXiv:2405.20038* 1 (1), pp. 1–7. Available at: <https://doi.org/10.48550/arXiv.2405.20038>.
- Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Pedreschi, D. and Giannotti, F. (2019). ‘A survey of methods for explaining black box models’. *ACM Computing Surveys* 51 (5). Comprehensive survey on explainability methods (often cited as Guidotti et al., 2018/2019). (Accessed: 19 September 2025), pp. 1–42. Available at: <https://doi.org/10.1145/3236009>.
- Guyon, I. and Elisseeff, A. (2003). ‘An introduction to variable and feature selection’. *Journal of Machine Learning Research* 3, pp. 1157–1182. URL: <https://www.jmlr.org/papers/volume3/guyon03a/guyon03a.pdf>.
- Guyon, I., Weston, J., Barnhill, S. and Vapnik, V. (2002). ‘Gene selection for cancer classification using support vector machines’. *Machine Learning* 46 (1), pp. 389–422. Available at: <https://doi.org/10.1023/A:1012487302797>. URL: <https://doi.org/10.1023/A:1012487302797>.
- Han, J., Kamber, M. and Pei, J. (2011). ‘Data Mining: Concepts and Techniques’. 3rd. Morgan Kaufmann.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). ‘The Elements of Statistical Learning: Data Mining, Inference, and Prediction’. 2nd. Springer. Available at: <https://doi.org/https://doi.org/10.1007/978-0-387-84858-7>. URL: <https://doi.org/10.1007/978-0-387-84858-7>.
- Hauke, J. and Kossowski, T. (2011). ‘Comparison of values of Pearson’s and Spearman’s correlation coefficients on the same sets of data’. *Quaestiones Geographicae* 30 (2), pp. 87–93.
- He, H. and Garcia, E. A. (2009). ‘Learning from imbalanced data’. *IEEE Transactions on Knowledge and Data Engineering* 21 (9). (Accessed: 19 September 2025), pp. 1263–1284. Available at: <https://doi.org/10.1109/TKDE.2008.239>. URL: <https://doi.org/10.1109/TKDE.2008.239>.

- Hochreiter, S. and Schmidhuber, J. (1997a). ‘Long Short-Term Memory’. *Neural Computation* 9 (8), pp. 1735–1780.
- (1997b). ‘Long short-term memory’. *Neural Computation* 9 (8). Original LSTM paper. (Accessed: 19 September 2025), pp. 1735–1780. Available at: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- HOPSWORKS Editors (2025). ‘Encoding (for Features)’. Accessed: 20 September 2025. URL: <https://www.hopsworks.ai/dictionary/encoding-for-features>.
- Houssel, R. B. P., Singh, P., Layeghy, S. and Portmann, M. (2024). ‘Towards explainable network intrusion detection using large language models’. *arXiv preprint arXiv:2408.04342* 1 (1), pp. 1–7. Available at: <https://doi.org/10.48550/arXiv.2408.04342>.
- ‘HTTP flood attack’ (2025). Accessed: 1 July 2025. URL: <https://www.cloudflare.com/en-gb/learning/ddos/http-flood-ddos-attack/>.
- imbalanced-learn developers (2023). ‘Imbalanced-learn Documentation’. Accessed: 28 August 2025.
- Inc., P. T. (2023). ‘Plotly Express Documentation’. Accessed: 28 August 2025.
- IntellectusConsulting Editors (2025). ‘Correlation (Pearson, Kendall, Spearman)’. Accessed: 20 September 2025. URL: <https://www.statisticssolutions.com/free-resources/directory-of-statistical-analyses/correlation-pearson-kendall-spearman/>.
- Ioffe, S. and Szegedy, C. (2015). ‘Batch normalization: Accelerating deep network training by reducing internal covariate shift’. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. (Accessed: 19 September 2025), pp. 448–456. URL: <http://arxiv.org/abs/1502.03167>.
- Jolliffe, I. T. and Cadima, J. (2016). ‘Principal component analysis: a review and recent developments’. *Philosophical Transactions of the Royal Society A* 374 (2065), pp. 20150202.
- Kaggle (n.d.[a]). ‘Easy way to download Kaggle data in Google Colab’. <https://www.kaggle.com/discussions/general/74235>. Accessed: 28 August 2025.
- (n.d.[b]). ‘Kaggle API Documentation (GitHub Repository)’. <https://github.com/Kaggle/kaggle-api>. Accessed: 28 August 2025.
- (n.d.[c]). ‘Public API – Kaggle’. <https://www.kaggle.com/docs/api>. Accessed: 28 August 2025.
- Karpathy, A. (2024). ‘Vibe coding is the future of LLM-based software engineering.’ X post, 14 April 2024. [Accessed 25 June 2025]. URL: Available%20at:%20<https://x.com/karpathy/status/1886192184808149383?lang=en>.
- Keras Team (2023). ‘Hyperparameter tuning with Keras Tuner’. Keras Documentation. (Accessed: 19 September 2025). URL: https://keras.io/guides/keras_tuner/getting_started/.
- Kingma, D. P. and Ba, J. (2015). ‘Adam: A Method for Stochastic Optimization’. *International Conference on Learning Representations*.
- Kiranyaz, S., Ince, T., Abdeljaber, O., Avci, O. and Gabbouj, M. (2021). ‘1D Convolutional Neural Networks and Applications: A Survey’. *Mechanical Systems and Signal Processing* 151, pp. 107398. Available at: <https://doi.org/https://doi.org/10.1016/j.ymssp.2020.107398>. URL: <https://doi.org/10.1016/j.ymssp.2020.107398>.
- Kirk, A. (2016). ‘Data Visualisation: A Handbook for Data Driven Design’. London: SAGE.
- Kohavi, R. (1995). ‘A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection’. In: *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*. Vol. 2, pp. 1137–1143. URL: <https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf>.
- Kuhn, M. and Johnson, K. (2013). ‘Applied Predictive Modeling’. New York: Springer.

- LeCun, Y., Bengio, Y. and Hinton, G. (2015). ‘Deep learning’. *Nature* 521 (7553). (Accessed: 19 September 2025), pp. 436–444. Available at: <https://doi.org/10.1038/nature14539>. URL: <https://doi.org/10.1038/nature14539>.
- Lemaître, G., Nogueira, F. and Aridas, C. K. (2017). ‘Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning’. *Journal of Machine Learning Research* 18 (17), pp. 1–5. URL: <http://jmlr.org/papers/v18/16-365.html>.
- Lin, L.-J. (1992). ‘Self-improving reactive agents based on reinforcement learning, planning and teaching’. *Machine Learning* 8 (3-4). (Accessed: 19 September 2025), pp. 293–321. Available at: <https://doi.org/10.1007/BF00992699>. URL: <https://doi.org/10.1007/BF00992699>.
- Little, R. J. A. and Rubin, D. B. (2019). ‘Statistical Analysis with Missing Data’. 3rd. Wiley. Available at: <https://doi.org/https://doi.org/10.1002/9781119482260>. URL: <https://doi.org/10.1002/9781119482260>.
- Liu, F. T., Ting, K. M. and Zhou, Z.-H. (2008). ‘Isolation Forest’. In: *2008 IEEE International Conference on Data Mining*, pp. 413–422.
- Liu, R. and Zou, J. (2017). ‘The Effects of Memory Replay in Reinforcement Learning’. Accessed: 20 September 2025. URL: <https://arxiv.org/abs/1710.06574#:~:text=Experience%20replay%20is%20a%20key,achieves%20consistently%20good%20empirical%20performance..>
- Lundberg, S. and Lee, S. (2017). ‘A Unified Approach to Interpreting Model Predictions’. [Accessed 27 June 2025]. URL: <https://arxiv.org/abs/1705.07874>.
- Lundberg, S. et al. (2017). ‘SHAP (SHapley Additive exPlanations)’. Python package: <https://github.com/slundberg/shap>.
- Marimuthu Parthiban (2024). ‘Dropout Regularization in Deep Learning’. Accessed: 20 September 2025. URL: <https://www.analyticsvidhya.com/blog/2022/08/dropout-regularization-in-deep-learning/#h-what-s-dropout>.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S. and Hassabis, D. (2015). ‘Human-level control through deep reinforcement learning’. *Nature* 518 (7540). (Accessed: 19 September 2025), pp. 529–533. Available at: <https://doi.org/10.1038/nature14236>. URL: <https://doi.org/10.1038/nature14236>.
- Molnar, C. (2025). ‘Interpretable Machine Learning: A Guide for Making Black Box Models Explainable’. 3rd ed. Available at: <https://christophm.github.io/interpretable-ml-book/> (Accessed: 19 September 2025). URL: <https://christophm.github.io/interpretable-ml-book/>.
- Moustafa, N. and Slay, J. (2015). ‘UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)’. In: *2015 Military Communications and Information Systems Conference (MilCIS)*. IEEE, pp. 1–6. Available at: <https://doi.org/10.1109/MilCIS.2015.7348942>. URL: <https://doi.org/10.1109/MilCIS.2015.7348942>.
- Ng, A. Y. (2004). ‘Feature selection, L1 vs. L2 regularization, and rotational invariance’.
- Niculescu-Mizil, A. and Caruana, R. (2005). ‘Predicting good probabilities with supervised learning’. In: *Proceedings of the 22nd International Conference on Machine Learning (ICML)*. (Accessed: 19 September 2025), pp. 625–632. Available at: <https://doi.org/10.1145/1102351.1102430>. URL: <https://doi.org/10.1145/1102351.1102430>.
- O’Malley, T., Bursztein, E., Long, J., Chollet, F., Jin, H., Invernizzi, L. et al. (2019). ‘Keras Tuner’. GitHub repository. (Accessed: 19 September 2025). URL: <https://github.com/keras-team/keras-tuner>.

- pandas development team (2023a). ‘pandas Documentation’. Accessed: 28 August 2025.
- (2023b). ‘pandas.read_csv — pandas Documentation’. Accessed: 28 August 2025.
- Pandey Swarnima (2020). ‘How to choose the size of the convolution filter or Kernel size for CNN?’ Accessed: 20 September 2025. URL: <https://medium.com/analytics-vidhya/how-to-choose-the-size-of-the-convolution-filter-or-kernel-size-for-cnn-86a55a1e2d15>.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2011). ‘Scikit-learn: Machine Learning in Python’. *Journal of Machine Learning Research* 12 (85), pp. 2825–2830. URL: <http://jmlr.org/papers/v12/pedregosa11a.html>.
- Platt, J. (1999). ‘Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods’. In: *Advances in Large Margin Classifiers*. (Accessed: 19 September 2025). MIT Press, pp. 61–74. URL: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=10.1.1.41.1639>.
- Powers, D. M. (2011). ‘Evaluation: From precision, recall and F-Measure to ROC, informedness, markedness & correlation’. *Journal of Machine Learning Technologies* 2 (1), pp. 37–63.
- Python Developer (2025). ‘NSL-KDD Intrusion Detection Dataset’. Accessed: 1 July 2025. URL: <https://www.kaggle.com/datasets/programmer3/nsl-kdd-intrusion-detection-dataset>.
- Rahman, M. A. et al. (2020). ‘A comparative study of different feature selection approaches for intrusion detection’. *Future Internet* 12 (11), pp. 190. Available at: <https://doi.org/https://doi.org/10.3390/fi12110190>. URL: <https://doi.org/10.3390/fi12110190>.
- Rakib Hossain Sajib (2019). ‘DDoS Detection using Machine Learning’. Accessed: 1 July 2025. URL: <https://www.kaggle.com/code/rakibhossainsajib/ddos-detection-using-machine-learning>.
- Ren, K., Zeng, Y., Zhong, Y., Sheng, B. and Zhang, Y. (2023). ‘MAFSIDS: a reinforcement learning-based intrusion detection model for multi-agent feature selection networks’. *Journal of Big Data* 10, pp. 137. Available at: <https://doi.org/10.1186/s40537-023-00814-4>.
- Repository, U. M. L. (1999). ‘KDD Cup 1999 Data Set Documentation’. <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>. Accessed: 28 August 2025.
- Ribeiro, M. T., Singh, S. and Guestrin, C. (2016a). ‘LIME: Local Interpretable Model-Agnostic Explanations’. Python package: <https://github.com/marcotcr/lime>.
- Ribeiro, T. M. ., Singh, S. and Guestrin, C. (2016b). ‘“Why Should I Trust You?” Explaining the Predictions of Any Classifier’. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. San Francisco, CA: Association for Computing Machinery DIGITAL LIBRARY, pp. 1135–1144. Available at: <https://doi.org/10.1145/2939672.2939778>.
- Ring, M., Wunderlich, S., Scheuring, D., Landes, D. and Hotho, A. (2019). ‘A survey of network-based intrusion detection data sets’. *Computers and Security* 86, pp. 147–167. Available at: <https://doi.org/https://doi.org/10.1016/j.cose.2019.06.005>. URL: <https://doi.org/10.1016/j.cose.2019.06.005>.
- Rodgers, J. L. and Nicewander, W. A. (1988). ‘Thirteen ways to look at the correlation coefficient’. *The American Statistician* 42 (1), pp. 59–66. Available at: <https://doi.org/https://doi.org/10.1080/00031305.1988.10475524>.

- Saito, T. and Rehmsmeier, M. (2015). ‘The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets’. *PLOS ONE* 10 (3). (Accessed: 19 September 2025), pp. e0118432. Available at: <https://doi.org/10.1371/journal.pone.0118432>. URL: <https://doi.org/10.1371/journal.pone.0118432>.
- Samek, W., Wiegand, T. and Müller, K.-R. (2017). ‘Explainable artificial intelligence: Understanding, visualizing and interpreting deep learning models’. *arXiv preprint arXiv:1708.08296*. URL: <https://arxiv.org/abs/1708.08296>.
- Saxena Shipra (2024). ‘Binary Cross Entropy/Log Loss for Binary Classification’. Accessed: 20 September 2025. URL: <https://www.analyticsvidhya.com/blog/2021/03/binary-cross-entropy-log-loss-for-binary-classification/>.
- scikit-learn developers (2023). ‘scikit-learn Documentation’. Accessed: 28 August 2025.
- Shafiq, M., Tian, Z., Bashir, A. K., Du, X. and Guizani, M. (2020). ‘IoT malicious traffic identification using wrapper-based feature selection mechanisms’. *Computers and Security* 94, pp. 101863. Available at: <https://doi.org/10.1016/j.cose.2020.101863>. URL: <https://doi.org/10.1016/j.cose.2020.101863>.
- Shapley, L. S. (1953). ‘A value for n-person games’. In: *Contributions to the Theory of Games, Volume II*. Ed. by Kuhn, H. W. and Tucker, A. W. Classic Shapley value formulation (also available in RAND tech reports). (Accessed: 19 September 2025). Princeton University Press, pp. 307–317.
- Sharafaldin, I., Lashkari, A. and Ghorbani, A. (2018). ‘Toward generating a new intrusion detection dataset and intrusion traffic characterization’. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116. Available at: <https://doi.org/10.5220/0006639801080116>.
- Sharafaldin, I., Lashkari, A. H. and Ghorbani, A. A. (2019). ‘Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy’. In: *Proceedings of the 6th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 331–340. Available at: <https://doi.org/https://doi.org/10.5220/0007368303310340>. URL: <https://doi.org/10.5220/0007368303310340>.
- Shearer, C. (2000). ‘The CRISP-DM model: The new blueprint for data mining’. *Journal of Data Warehousing* 5 (4), pp. 13–22.
- Shen, S., Cai, C., Li, Z., Shen, Y., Wu, G. and Yu, S. (2024). ‘Deep Q-network-based heuristic intrusion detection against edge-based SIoT zero-day attacks’. *Applied Soft Computing* 150, pp. 111080. Available at: <https://doi.org/10.1016/j.asoc.2023.111080>.
- Shiravi, A., Shiravi, H., Tavallaei, M. and Ghorbani, A. A. (2012). ‘Toward developing a systematic approach to generate benchmark datasets for intrusion detection’. *Computers and Security* 31 (3), pp. 357–374. Available at: <https://doi.org/https://doi.org/10.1016/j.cose.2011.12.012>. URL: <https://doi.org/10.1016/j.cose.2011.12.012>.
- Sievert, C. (2020). ‘Interactive Web-Based Data Visualization with R, Plotly, and Shiny’. 1st. Chapman and Hall/CRC.
- solarmainframe (2018). ‘IDS 2018 Intrusion CSVs (CSE-CIC-IDS2018)’. Accessed: 1 July 2025. URL: <https://www.kaggle.com/datasets/solarmainframe/ids-intrusion-csv>.
- Sommer, R. and Paxson, V. (2010). ‘Outside the closed world: On using machine learning for network intrusion detection’. In: *Proceedings of the IEEE Symposium on Security and Privacy (SandP)*, pp. 305–316. Available at: <https://doi.org/https://doi.org/10.1109/SP.2010.25>. URL: <https://doi.org/10.1109/SP.2010.25>.
- Song, X. and Ma, Q. (2024). ‘Intrusion Detection using Federated Attention Neural Network for Edge Enabled Internet of Things’. *Journal of Grid Computing* 22 (15), pp. 1–17. Available at: <https://doi.org/10.1007/s10723-023-09725-3>.

- Song, Y., Xiong, W., Zhu, D., Wu, W., Qian, H., Song, M., Huang, H., Li, C., Wang, K., Yao, R., Tian, Y. and Li, S. (2023). ‘RestGPT: Connecting Large Language Models with Real-World RESTful APIs’. arXiv preprint arXiv:2306.06624. arXiv:2306.06624. (Accessed: 19 September 2025). URL: <https://arxiv.org/abs/2306.06624>.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014). ‘Dropout: A Simple Way to Prevent Neural Networks from Overfitting’. *Journal of Machine Learning Research* 15, pp. 1929–1958.
- Strickland, C., Zakar, M., Saha, C., S., S. N., Tasnim, N., Lizotte, D. J. and Haque, A. (2024). ‘DRL-GAN: A hybrid approach for binary and multiclass network intrusion detection’. *Sensors* 24 (9), pp. 2746. Available at: <https://doi.org/10.3390/s24092746>.
- Strickland, K., Sharma, N. and Xu, H. (2024). ‘Hybrid Deep Reinforcement Learning for Intrusion Detection: A Comparative Study’. *IEEE Transactions on Information Forensics and Security* 19, pp. 1–12. Available at: <https://doi.org/10.1109/TIFS.2024.1001234>.
- Sutton, R. S. and Barto, A. G. (2018). ‘Reinforcement Learning: An Introduction’. 2nd. (Accessed: 19 September 2025). Cambridge, MA: MIT Press. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- Tavallae, M., Bagheri, E., Lu, W. and Ghorbani, A. (2009a). ‘A detailed analysis of the KDD CUP 99 data set’. In: *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. IEEE, pp. 1–6. Available at: <https://doi.org/10.1109/CISDA.2009.5356528>.
- Tavallae, M., Bagheri, E., Lu, W. and Ghorbani, A. A. (2009b). ‘A detailed analysis of the KDD CUP 99 data set’. In: *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pp. 1–6. Available at: <http://doi.org/10.1109/CISDA.2009.5356528>. URL: <https://doi.org/10.1109/CISDA.2009.5356528>.
- Team, K. (2023a). ‘Keras Tuner Documentation’. Accessed: 28 August 2025.
- Team, S. (2023b). ‘SciKeras Documentation’. Accessed: 28 August 2025.
- Team, T. (2023c). ‘TensorFlow Documentation’. Accessed: 28 August 2025.
- Tran, L.-A., Nguyen Tran, C., Park, D.-C., Carrabina, J. and Castells-Rufas, D. (2024). ‘Toward Improving Robustness of Object Detectors Against Domain Shift’. *arXiv preprint arXiv:2403.12049*. Available at: [arXiv.org](https://arxiv.org/abs/2403.12049). Available at: <https://doi.org/10.48550/arXiv.2403.12049>. URL: <https://doi.org/10.48550/arXiv.2403.12049>.
- Tukey, J. W. (1977). ‘Exploratory Data Analysis’. Addison-Wesley.
- ‘UDP flood attack’ (2025). Accessed: 1 July 2025. URL: <https://www.cloudflare.com/en-gb/learning/ddos/glossary/user-datagram-protocol-udp/>.
- Ullah, I. and Mahmoud, Q. H. (2020). ‘An effective feature selection technique for cyber intrusion detection systems using XGBoost’. *Computers and Security* 96, pp. 101879. Available at: <https://doi.org/10.1016/j.cose.2020.101879>. URL: <https://doi.org/10.1016/j.cose.2020.101879>.
- Vishwakarma Swapnil (2024). ‘How to Understand Sigmoid Function in Artificial Neural Networks?’ Accessed: 20 September 2025. URL: <https://www.analyticsvidhya.com/blog/2023/01/why-is-sigmoid-function-important-in-artificial-neural-networks/#h-understanding-the-sigmoid-function>.
- Waskom, M., Botvinnik, O., O’Kane, D., Hobson, P., Lukauskas, S., Gemperline, D. C., Augspurger, T., Halchenko, Y. O., Cole, J. B., Warmenhoven, J., Ruiter, J. de, Pye, C., Hoyer, S., Vanderplas, J., Villalba, S., Kunter, G., Quintero, E., Bachant, P., Martin, M., Meyer, K., Miles, A., Ram, K., Yarkoni, T., Williams, M. L., Evans, C., Fitzgerald, C., Brian and Yaroslav, H. (2017). ‘Seaborn: Statistical Data Visualization’. *Journal of Open*

- Source Software 2* (11), pp. 302. Available at: <https://doi.org/https://doi.org/10.21105/joss.0302>.
- Watkins, C. J. C. H. and Dayan, P. (1992). ‘Q-learning’. *Machine Learning 8*, pp. 279–292.
- ‘What is a SYN flood attack?’ (2025). Accessed: 1 July 2025. URL: <https://www.cloudflare.com/en-gb/learning/ddos/syn-flood-ddos-attack/>.
- ‘what is u2r attack’ (2025). Accessed: 1 July 2025. URL: https://www.google.com/search?q=what+is+u2r+attack&sca_esv=af2334d4398b9d42&ei=Wmtka0v_Dsn0i-gPq_3SmQY&ved=0ahUKEwjrlMe45Jy0AxVJ-gIHHau-NGMQ4dUDCBI&uact=5&oq=what+is+u2r+attack&gs_lp=Egxnd3Mtd2l6LXNlcniAiEndoYXQgaXMgdTJyIGF0dGFjazIGEAAyFhgeMgsQABiABBiGAXiKBTILEAAyGAYhgMYigUyBRAAG08FMgUQABjvBUjBD1CIBliZDnACeACQAQCYAWagAacEqgEDNi4xuAEDyAEA-AEBmAIIoAL-A8ICChAAGLADGNyEGEfCaggQABiABBiBJgDAIgGAZAGAPIHazcuMaAHpRayBwM1LjG4B_MDwgcDMi04yAcn&scient= gws-wiz-serp.
- ‘What is UDP?’ (2025). Accessed: 1 July 2025. URL: <https://www.cloudflare.com/en-gb/learning/ddos/glossary/user-datagram-protocol-udp/>.
- Wirth, R. and Hipp, J. (2000). ‘CRISP-DM: Towards a standard process model for data mining’. In: *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining (PADKDD)*, pp. 29–39.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M. and Brew, J. (2020). ‘Transformers: State-of-the-art Natural Language Processing’. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Hugging Face / Transformers library paper/demo — use when citing the Transformers toolkit. (Accessed: 19 September 2025). Association for Computational Linguistics, pp. 38–45. URL: <https://www.aclweb.org/anthology/2020.emnlp-demos.6/>.
- Wu, H. and Gu, X. (2015). ‘Towards Dropout Training for Convolutional Neural Networks’. *arXiv preprint arXiv:1512.02423*. Available at: arXiv.org. URL: <https://arxiv.org/abs/1512.02423>.
- Xi, C., Wang, H. and Wang, X. (2024). ‘A novel multi-scale network intrusion detection model with transformer’. *Scientific Reports 14*, pp. 23239. Available at: <https://doi.org/10.1038/s41598-024-74214-w>.
- Yin, C.-l., Zhu, Y.-f., Fei, J.-l. and He, X.-z. (2017). ‘A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks’. *IEEE Access 5*, pp. 21954–21961. Available at: <https://doi.org/10.1109/ACCESS.2017.2762418>. URL: <https://doi.org/10.1109/ACCESS.2017.2762418>.
- Zhang, Y., Ravie, C. M. and Qamar, F. (2023). ‘A review of deep learning applications in intrusion detection systems: Overcoming challenges in spatiotemporal feature extraction and data imbalance’. *Applied Sciences 15* (3), pp. 1552. Available at: <https://doi.org/10.3390/app15031552>.